

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

웹 애플리케이션과 Vue.js



한국기술교육대학교
온라인평생교육원

학습내용

- 프론트엔드 웹 애플리케이션
- 자바스크립트 프레임워크와 Vue.js

학습목표

- **프론트엔드 웹 애플리케이션**의 개념 및 구현 기술에 대해 설명할 수 있다.
- **자바스크립트 프레임워크**의 목적과 종류에 대해 설명할 수 있다.
- **Vue.js**의 특징에 대해 설명할 수 있다.

프론트엔드 웹 애플리케이션

프론트엔드 웹 애플리케이션

1 프론트엔드 웹 애플리케이션

- 서버 사이트의 도움을 받지 않고도 스스로 무언가를 해결하고 제공할 수 있는 서비스를 가진 웹(Web)
- 서버 사이트와 적절하게 연동하면서 기존에 서버 사이트에서 결정했던 많은 기능적인 측면이 클라이언트 사이트에서 구현되는 웹

프론트엔드 웹 애플리케이션

1 프론트엔드 웹 애플리케이션

- CS 프로그램 환경처럼 독립적인 클라이언트 사이트 앱(App)인데 단지 실행환경이 웹으로 개발

프론트엔드 웹 애플리케이션

프론트엔드 웹 애플리케이션

2 웹 애플리케이션의 진화

기존의 애플리케이션(Application)

서버 사이트의 기능적인 측면 강조



현재의 웹 앱

클라이언트 사이트의 기능적인 측면 강조

프론트엔드 웹 애플리케이션

2 웹 애플리케이션의 진화

서버 사이트
중심의 웹

➤
2000년대
중반

웹 2.0의
리치 클라이언트
(Rich Client) 강조

프론트엔드 웹 애플리케이션

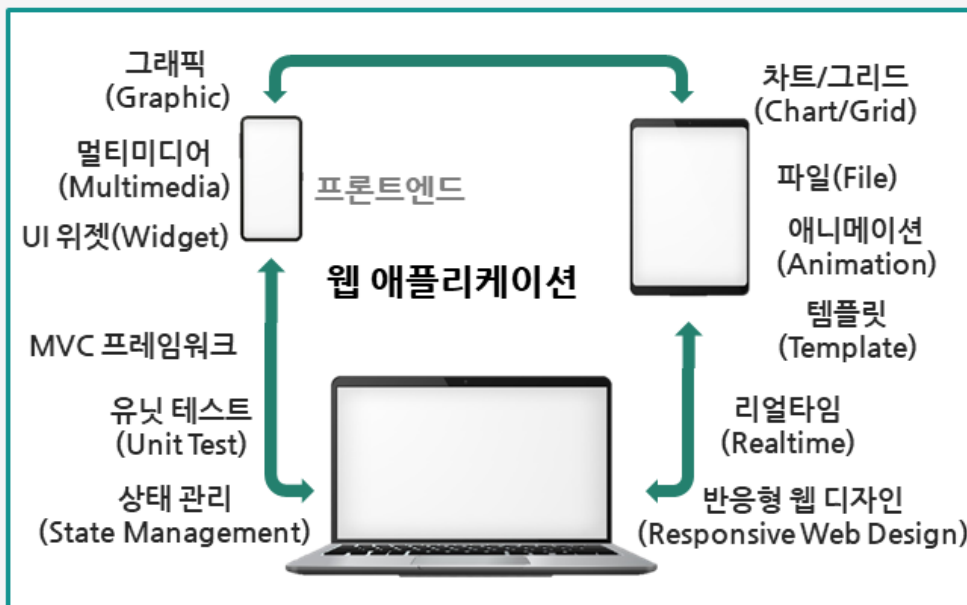
프론트엔드 웹 애플리케이션

2 웹 애플리케이션의 진화

- 화면만 화려한 클라이언트 사이드(Client Side)가 아닌
일정 정도는 **애플리케이션 기능이 가능한**
클라이언트 사이드 웹 애플리케이션의 요구가 발생하였음
 - Ajax, Flex, SilverLight 같은 기술이 탄생하게 됨

프론트엔드 웹 애플리케이션

3 프론트엔드 웹 애플리케이션 기술요소



프론트엔드 웹 애플리케이션

프론트엔드 웹 애플리케이션

4 HTML5와 자바스크립트 프레임워크

HTML5 적용

- 애플리케이션으로서의 다양한 기능 구현 필요

자바스크립트 프레임워크 적용

- 애플리케이션의 Base Architecture 필요

프론트엔드 웹 애플리케이션

5 HTML5 기술요소

1

web socket

... 연결 지향형 socket 통신

2

web storage

... 데이터 영속적 저장

3

index db

... 데이터 영속적 저장

프론트엔드 웹 애플리케이션

프론트엔드 웹 애플리케이션

5 HTML5 기술요소

4

canvas

→ Graphic 요소

5

audio, video

→ Multimedia 데이터 처리

6

Responsive Web Design

→ 다양한 사이즈의 Device를 위한 Design

프론트엔드 웹 애플리케이션

6 자바스크립트 프레임워크

Angular



React.js



Vue.js



자바스크립트 프레임워크와 Vue.js

자바스크립트 프레임워크와 Vue.js

1 자바스크립트 프레임워크

- 자바스크립트를 이용한 웹 애플리케이션의 기본 Architecture 제공을 목적으로 함

UI
Component

동적 재사용 가능한 UI Widget

Router

Hash Control과 이를 통한
SPA 구현

State
Management
Framework

앱의 상태 정보 유지 프레임워크

자바스크립트 프레임워크와 Vue.js

2 AngularJS

- 2009년 Misko hevery와 Adam abrons의 공동 개발
- MIT 라이선스, Open Source 라이브러리
- 2013년 9월 정식버전 출시(Angular1.x)

자바스크립트 프레임워크와 Vue.js

자바스크립트 프레임워크와 Vue.js

2 AngularJS

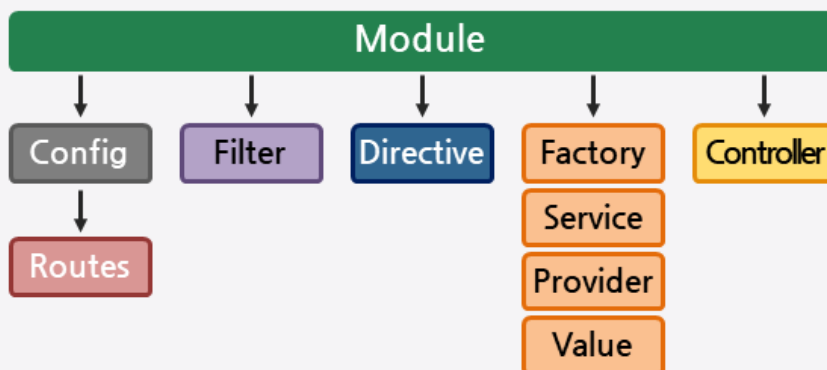
- MVW Pattern (Model - View - Whatever)



자바스크립트 프레임워크와 Vue.js

2 AngularJS

- Two-Way Data Binding



자바스크립트 프레임워크와 Vue.js

자바스크립트 프레임워크와 Vue.js

3 Angular

- 2014년 10월 ng-conference에서 발표
- 전체 프레임워크를 재작성한 것이며 1.x 버전과의 단절적인 변화

자바스크립트 프레임워크와 Vue.js

3 Angular

- Develop Across All Platforms

For Web

Mobile Web

Native Mobile

Native Desktop

자바스크립트 프레임워크와 Vue.js

자바스크립트 프레임워크와 Vue.js

3 Angular

- 1.x와 비교 시 성능개선 및 개발 편의성을 위한 Architecture 재정의
- TypeScript, TypeScript 필수는 아니며 Dart, ECMAScript 6 사용 가능

자바스크립트 프레임워크와 Vue.js

4 React.js

- Facebook의 View 부분을 위한 컴포넌트 라이브러리

UI Component

VIRTUAL DOM

Unidirectional Data Flow

JSX

자바스크립트 프레임워크와 Vue.js

자바스크립트 프레임워크와 Vue.js

4 React.js

- Facebook의 View 부분을 위한 컴포넌트 라이브러리

Redux

Isomorphic JavaScript

React Native



React

자바스크립트 프레임워크와 Vue.js

5 Vue.js



Vue.js 특징

- 'Evan you'에 의해 개발 - 2013/12
- 웹 UI를 빠르게 작성하기 위한 목적

자바스크립트 프레임워크와 Vue.js

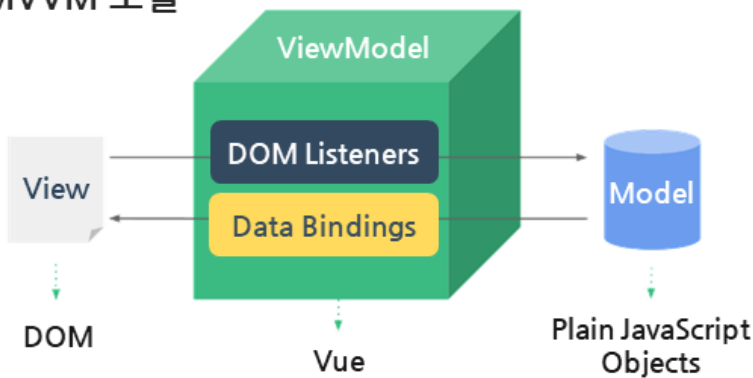
자바스크립트 프레임워크와 Vue.js

5 Vue.js



Vue.js 특징

- MVVM 모델



[이미지 출처 : vuejs.org]

자바스크립트 프레임워크와 Vue.js

5 Vue.js



Vue.js 특징

- 가상 DOM 지원
- Router
- Vuex

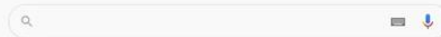
실습 플러스

웹 애플리케이션과 Vue.js

실습 플러스

1 Visual Studio Code Install

■ VSCode 1.56.1 버전 사용



Google 검색

I'm Feeling Lucky

대한민국

Google 정보 광고 비즈니스 검색의 권리

개인정보처리방침 약관

실습단계

Vue.js 실습 개발환경 구축

코드(Code)를 타이핑할 수 있는 에디터 설치

Vue.js로 애플리케이션 개발 시 주력으로 타이핑하는 코드는 자바스크립트와 HTML

개발자가 자바스크립트와 HTML을 타이핑하기 편한 에디터를 취사 선택하여 설치

VS Code에서 Vue.js 프로젝트 실습 진행

검색하여 간단하게 VS Code를 다운로드 받을 수 있는 사이트 접속

첫 번째 링크를 클릭하여 다운로드 버튼 클릭

Visual Studio Code = VS Code

기본적으로 동의한 후, 설치 진행

기본으로 디렉토리 선택되어 있음, 다른 디렉토리 선택도 가능함

시작 메뉴 폴더 선택의 기본값 그대로 유지

바탕화면 바로가기 아이콘 추가 설정

기본값으로 체크된 상태를 유지하여 종료 버튼 누르고 VS Code 실행

작업을 위해 개발자 임의의 workspace 디렉토리 제작

실습 플러스

실습단계
workspace 디렉토리는 VS Code에서 개발자가 타이핑했던 코드가 저장되는 위치
개발자가 임의의 폴더를 정해주면 됨
C:에 하나의 폴더 제작
File → Open Folder
작업한 것이 없기 때문에 비어있는 상태
Extensions : Vue.js를 이용하면서 추가적으로 설치할 수 있는 플러그인 설치 가능 메뉴
검색하여 다양한 플러그인 확인 가능

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Vue.js 개발환경



한국기술교육대학교
온라인평생교육원

학습내용

- Vue.js 설치
- Vue 애플리케이션 구조

학습목표

- Vue를 위한 개발환경을 구축할 수 있다.
- Vue 프로젝트의 구조에 대해 설명할 수 있다.

Vue.js 설치

Vue.js 설치

1 Node.js 소개



Node.js

- 크롬(Chrome) V8 자바스크립트(JavaScript) 엔진(Engine)으로 빌드된 자바스크립트 런타임(Runtime)
- 자바스크립트로 작성된 애플리케이션을 Node.js로 실행시킬 수 있음

Vue.js 설치

1 Node.js 소개



Node.js

- 자바스크립트 런타임이며 다양한 모듈(Module)을 설치해 개발함



Vue.js 설치

Vue.js 설치

2 Node.js Download 및 Install

다운로드 사이트 : <https://nodejs.org/>

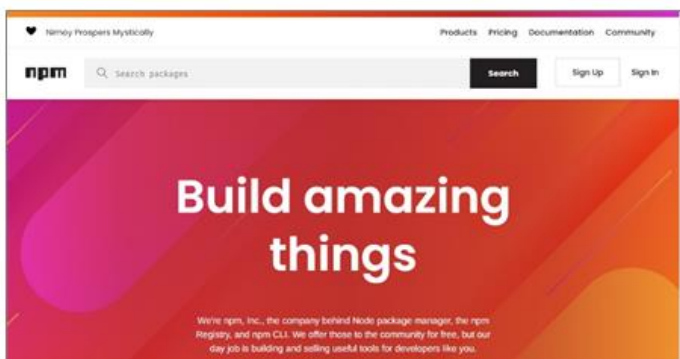


Vue.js 설치

3 npm repository

<https://www.npmjs.com/>

→ 다양한 자바스크립트 모듈(혹은 라이브러리)이 등록되어 있는 곳



Vue.js 설치

Vue.js 설치

4 npm CLI

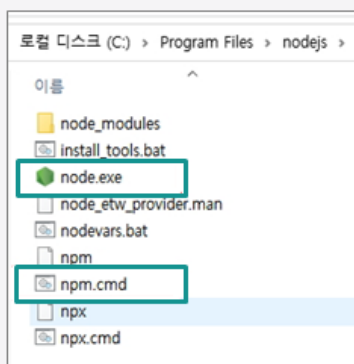
- npm CLI(Command Line Interface)명령어로 npm repository의 모듈을 다운로드 및 관리함

node.exe

자바스크립트 실행 명령어

npm.cmd

npm CLI 명령어



```
> npm install jquery
```

Vue.js 설치

5 Vue CLI 설치

- Vue 프로젝트 생성, 실행, 빌드 등 다양한 명령어를 제공하는 CLI 설치

```
> npm install -g @vue/cli
```


Vue.js 설치

Vue.js 설치

6 Vue 프로젝트(Project)

① Vue 프로젝트 생성

→ Vue CLI 명령어로 프로젝트 생성

```
> vue create vue-test
```

Vue.js 설치

6 Vue 프로젝트(Project)

② 프로젝트 실행

→ 프로젝트 폴더에서 npm 명령으로 실행

→ 프로젝트 자체에서 개발용 경량 서버 제공

→ Hot Deploy 제공

Vue.js 설치

Vue.js 설치

6 Vue 프로젝트(Project)

2 프로젝트 실행

```
>cd vue-test
>npm run serve
```



Vue.js 설치

6 Vue 프로젝트(Project)

3 프로젝트 빌드(build)

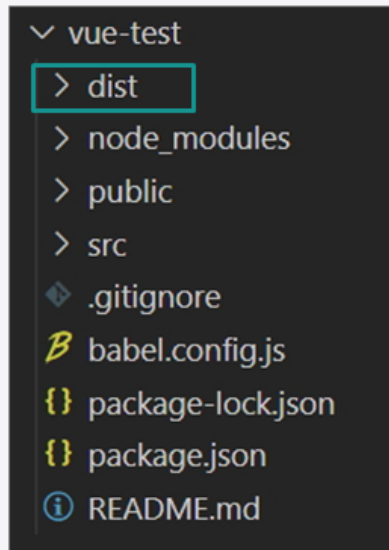
- ... 개발 완료 후 배포 파일로 빌드
- ... dist 폴더에 배포 파일 생성

```
>npm run build
```

Vue 애플리케이션 구조

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



dist

build 파일

node_modules

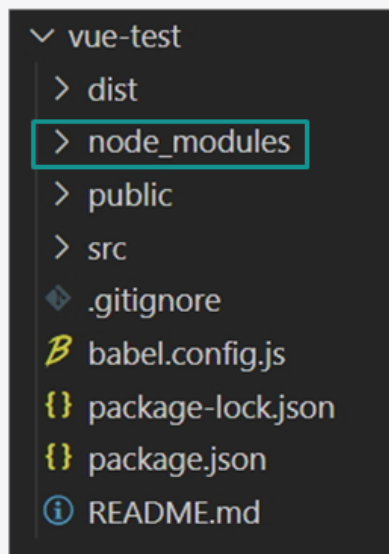
프로젝트를 위한 node module

public

공용 파일 위치
(html, css, icon image 등)

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



dist

build 파일

node_modules

프로젝트를 위한 node module

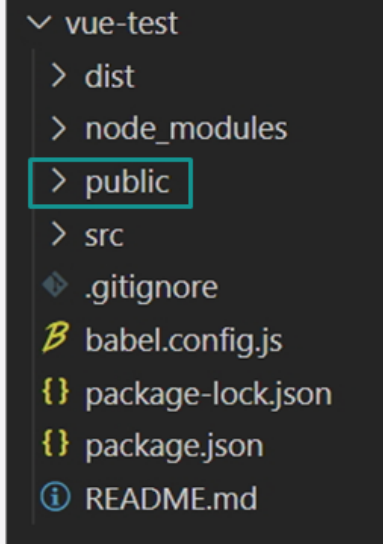
public

공용 파일 위치
(html, css, icon image 등)

Vue 애플리케이션 구조

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



dist

build 파일

node_modules

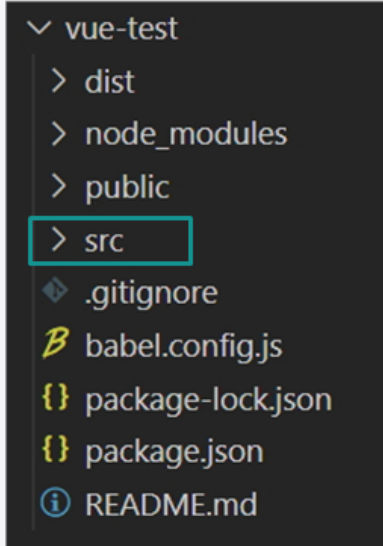
프로젝트를 위한 node module

public

공용 파일 위치
(html, css, icon image 등)

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



src

app 소스 파일

babel.config.js

babel 환경파일

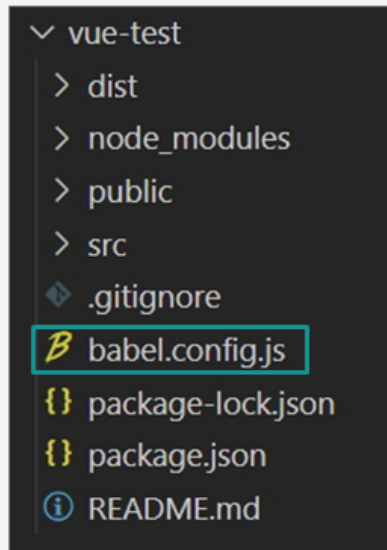
package.json

npm 환경파일

Vue 애플리케이션 구조

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



src

app 소스 파일

babel.config.js

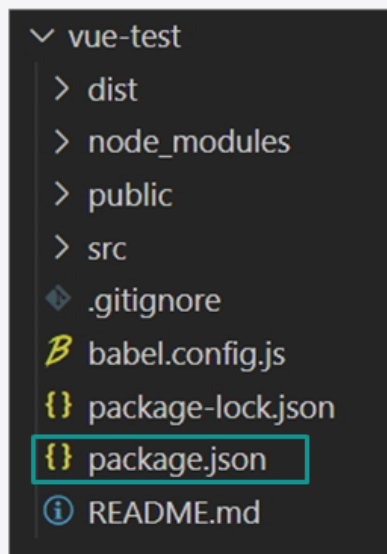
babel 환경파일

package.json

npm 환경파일

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



src

app 소스 파일

babel.config.js

babel 환경파일

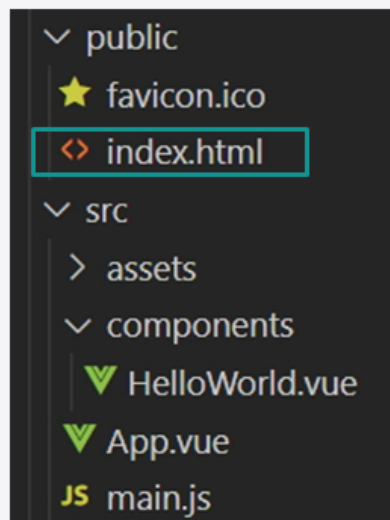
package.json

npm 환경파일

Vue 애플리케이션 구조

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



public/index.html

app entry point

src/main.js

Javascript entry point

src/App.vue

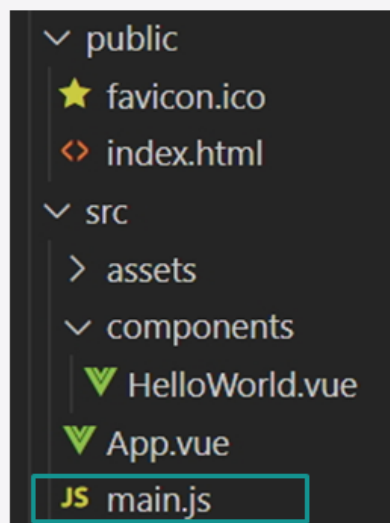
Component entry point

src/components/
HelloWorld.vue

Component

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



public/index.html

app entry point

src/main.js

Javascript entry point

src/App.vue

Component entry point

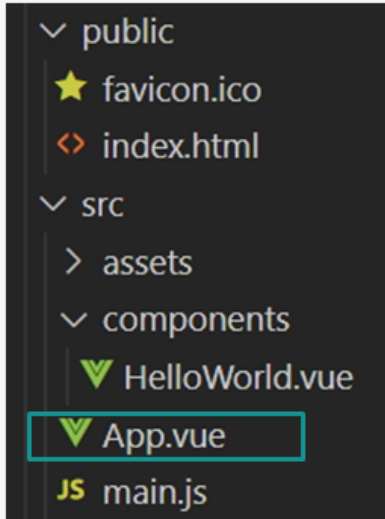
src/components/
HelloWorld.vue

Component

Vue 애플리케이션 구조

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



public/index.html app entry point

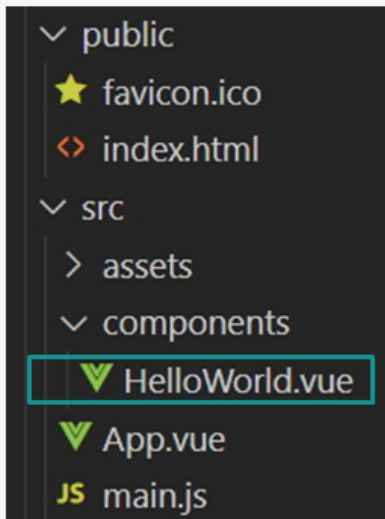
src/main.js Javascript entry point

src/App.vue Component entry point

src/components/
HelloWorld.vue Component

Vue 애플리케이션 구조

1 Vue 프로젝트 구조



public/index.html app entry point

src/main.js Javascript entry point

src/App.vue Component entry point

src/components/
HelloWorld.vue Component

Vue 애플리케이션 구조

Vue 애플리케이션 구조

2 index.html

- 브라우저에서 실행되는 App의 Entry Point
- `<div id="app"></div>` 위치에 Vue에 의한 결과 출력

```
<!DOCTYPE html>
<html lang="en">
  .....
  <body>
    .....
    <div id="app"></div>
  </body>
</html>
```

Vue 애플리케이션 구조

3 main.js

- HTML에 의해 로딩되어 실행되는 자바스크립트 Entry Point

```
import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```


Vue 애플리케이션 구조

Vue 애플리케이션 구조

4 App.vue

- Vue의 Component를 구성하기 위한 파일
- 확장자를 .vue로 작성

Vue 애플리케이션 구조

4 App.vue

```
<template>
  <HelloWorld msg="Welcome to Your Vue.js App"/>
</template>
```

Vue 애플리케이션 구조

Vue 애플리케이션 구조

4 App.vue

```
<script>
import HelloWorld from './components/HelloWorld.vue'

export default {
  name: 'App',
  components: {
    HelloWorld
  }
}
</script>

<style>
</style>
```

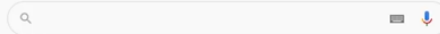
실습 플러스

Vue.js 개발환경

실습 플러스

1 Node.js 설치

■ VSCode 1.56.1 버전 사용



Google 검색

I'm Feeling Lucky

Node.js 설치 및 npm 명령으로 Vue.js 설치

실습단계

Node.js 설치 및 npm 명령으로 Vue.js 설치

Vue.js 명령어를 이용하여 Vue.js Project 생성

Node.js 설치

Nodejs.org 사이트 접속

Node.js 설치의 기본 값으로 진행해도 됨

Node.js가 설치되는 디렉토리 경로, 변경 가능

Install 종료

Node.js에서 제공하는 npm 명령으로 Vue의 CLI 명령어 설치

Vue 개발을 위한 일종의 툴!

Vue의 CLI 명령을 Node의 npm 명령으로 설치

CLI 설치의 특정 폴더에 하지 않아도 됨

Node에서 제공되고 있는 명령어 npm
: npm repository 서버로부터 다양한 것을 다운로드 가능Install
: 새로운 것을 설치

개인정보처리방침

약관

Google

광고

비즈니스

검색의 원리

Google 정보

대한민국

16

실습 플러스

실습단계

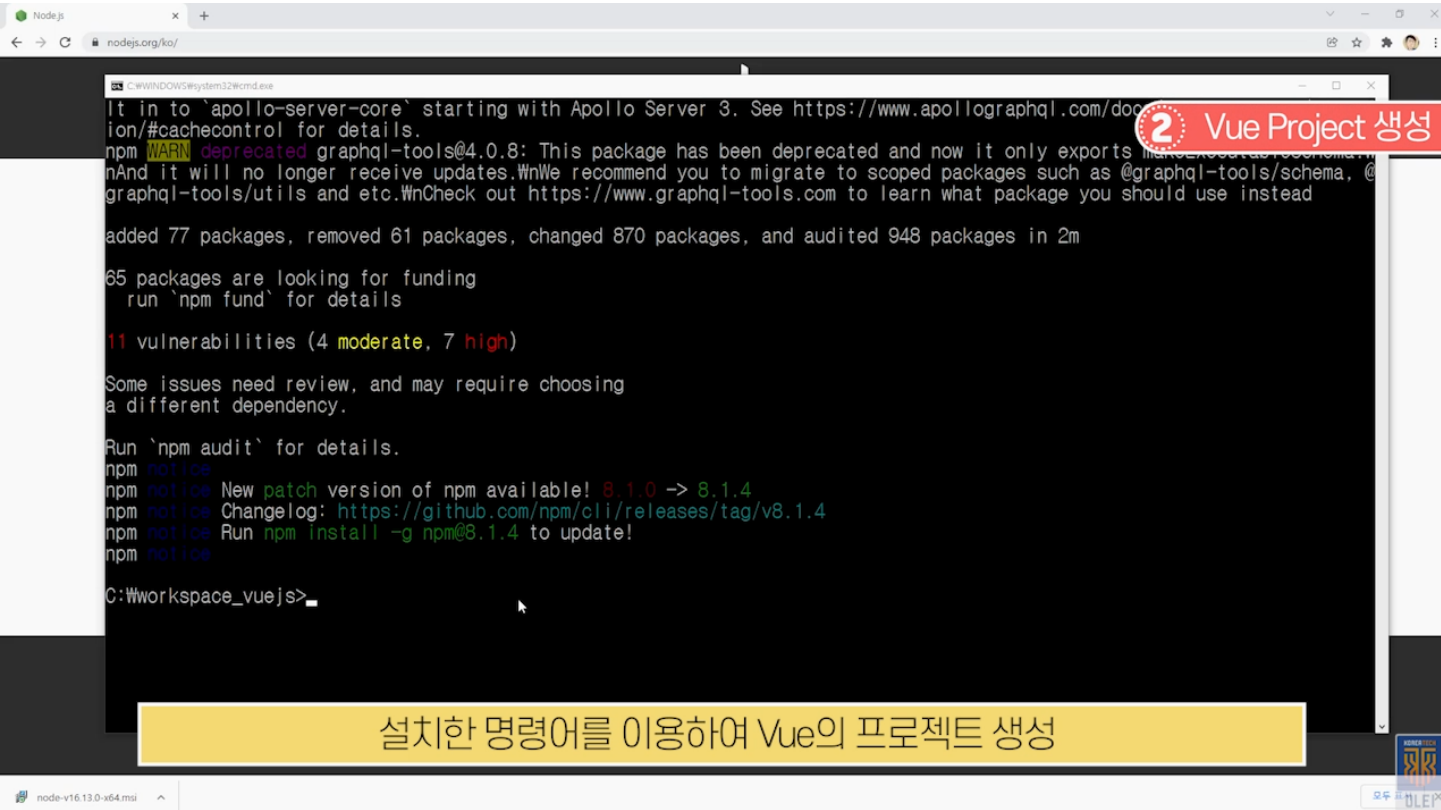
-g

: 명령 내리는 폴더의 서브에서만 사용, 글로벌로 올려서 사용할 것인가의 여부

npm repository 서버에 등록되어 있는 이름 작성

Vue의 CLI 명령어 설치 완료

실습 플러스



실습단계
설치한 명령어를 이용하여 Vue의 프로젝트 생성
vue 명으로 프로젝트 생성
어느 버전을 이용할 것인가?
방향키를 눌러서 원하는 버전 선택
방향키로 조작 후 엔터
프로젝트 생성 완료
생성된 프로젝트 폴더에 들어가 해당 명령어로 프로젝트 실행
프로젝트 폴더로 이동
npm 명령어로 서버 실행
Build 완료, 서버 구동된 상태
프로젝트가 정상적으로 동작하는지 브라우저로 테스트
브라우저 창에서 URL 입력
브라우저에서 프로젝트 실행
개발환경 구축 성공

실습 플러스

실습단계
vue-test로 만들어진 프로젝트와 하위 폴더 파일들을 볼 수 있음
src → App.vue에 의해서 components → HelloWorld 등이 나와있음
브라우저에 나타난 화면은 HelloWorld.vue 파일에서 제공
Hot Deploy : 연동 후 코드를 수정하고 저장 시 별도의 명령 없이도 바로 반영되는 기능
동작하고 있는 서버를 끊기 위해 Ctrl + C(서버 종료)
개발 완료 후 최종 배포를 위한 Build 진행
Build 진행 명령어 작성
Build 명령을 내리면 Build 된 내용이 dist 폴더에 들어감
개발 환경 구축을 위한 Node.js 설치 및 npm 명령으로 Cli 명령어 설치
Vue.js 명령어를 이용하여 Vue.js Project 생성 및 구동, Hot Deploy와 Build 확인

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

ES2015



한국기술교육대학교
온라인평생교육원

학습내용

- ECMAScript Overview
- Babel

학습목표

- **ECMAScript**에 대해 설명할 수 있다.
- **Babel의 역할**에 대해 설명할 수 있다.

ECMAScript Overview

ECMAScript Overview

1 Overview

① 자바스크립트(Javascript) Overview

자바스크립트 Overview

- 1995년 넷스케이프에 의해 개발된 언어
- 자바스크립트는 브라우저에서 실행되는 HTML 페이지에 언어적 기능을 제공하기 위해 개발됨
- 간단한 연산, 유저 입력 데이터 유효성 검증 등이 주 역할
- 스크립트 언어이며 브라우저의 자바스크립트 엔진에 의해 해석되어 실행됨

ECMAScript Overview

1 Overview

② ECMAScript Overview

ECMAScript Overview

- ECMAScript 혹은 ES로 불림
- ECMA International에 의해 표준이 정의되는 언어
- ECMA-254 기술 규격에 따라 정의된 표준화된 스크립트 프로그래밍 언어임
- 자바스크립트를 표준화 하기 위해 만들어 졌음

ECMAScript Overview

ECMAScript Overview

2 ECMAScript 초기 버전

ES3

- 1999년 발표
- 초창기 자바스크립트
- IE8 시절의 자바스크립트

ES5

- 2009년 발표
- 엄격한 오류 검사를 위한 Strict mode 추가
- forEach, map, filter 등의 Collection 관련 함수 추가

ECMAScript Overview

3 자바스크립트의 역할 변화

- 브라우저에서 실행되는 웹 애플리케이션 개발 필요성
- 백엔드(Back-End) 웹 애플리케이션 개발 언어로 진화
- 자바스크립트를 대체할 목적의 다양한 언어 등장

Typescript

Dart

Coffee
Script

ECMAScript Overview

ECMAScript Overview

4 ECMAScript 진화

- 대규모 자바스크립트 애플리케이션 개발을 위한 다양한 기능을 제공하는 정식의 언어로 진화

ECMAScript Overview

4 ECMAScript 진화

1 ES6(ES2015)

- 2015년 발표
- 블록단위 스코프 제공
- 모듈화 지원
- Template Literal 지원

ECMAScript Overview

ECMAScript Overview

4 ECMAScript 진화

2

ES7(ES2016)

3

ES8(ES2017)

4

ES9(ES2018)

5

ES10(ES2019)

Babel

Babel

1 ECMA Script 문제점

1

ECMAScript의 모든 기능이 모든 브라우저에서 정상적으로 실행되지 않음

2

ECMAScript는 표준이며 ECMAScript로 작성된 코드를 실행시켜 주는 것은 브라우저의 자바스크립트 엔진임

Babel

1 ECMA Script 문제점

● ES2015

```
(function () {
  function some(x, y = 10, z = 20) {
    console.log(`x=${x}, y=${y}, z=${z}`)
  }
  some(1)//x=1, y=10, z=20
  some(1, 2)//x=1, y=2, z=20
  some(1, 2, 3)//x=1, y=2, z=3
})();
```

Babel

Babel

1 ECMA Script 문제점

ES2015

Chrome	정상 실행
Edge	정상 실행
IE 11	에러

Babel

1 ECMA Script 문제점

ES2015

```
import { data1, funA, Shape } from './module';
```

Chrome	에러
Edge	에러
IE 11	에러

Babel

Babel

2 Babel

1 Babel

- Babel is a Javascript Compiler
- Use next generation Javascript, today



Babel

2 Babel

1 Babel

- 개발자가 작성한 파일을 브라우저의 자바스크립트 엔진이 해석할 수 있는 **자바스크립트 코드로 변경**
- 자바스크립트 Compiler(Transpiler로 불리기도 함)
- ES2015 추가된 자바스크립트를 대부분의 브라우저에서 지원하는 **ES5로 변형**이 주목적임
- ECMAScript 이외에 웹에서 이용되는 다양한 형태의 코드를 **브라우저에서 인지할 수 있는 코드로 변형해 줌**

Babel

Babel

2 Babel

② Babel 설치

```
npm install --save-dev @babel/core @babel/cli @babel/preset-env
```

Babel

2 Babel

③ Babel 환경파일 작성

→ babel.config.json 혹은 babel.config.js

```
const presets = [
  [
    "@babel/env"
  ],
];

module.exports = { presets };
```


Babel

Babel

2 Babel

4 Babel Compile

```
node_modules\.bin\babel src --out-dir build
```

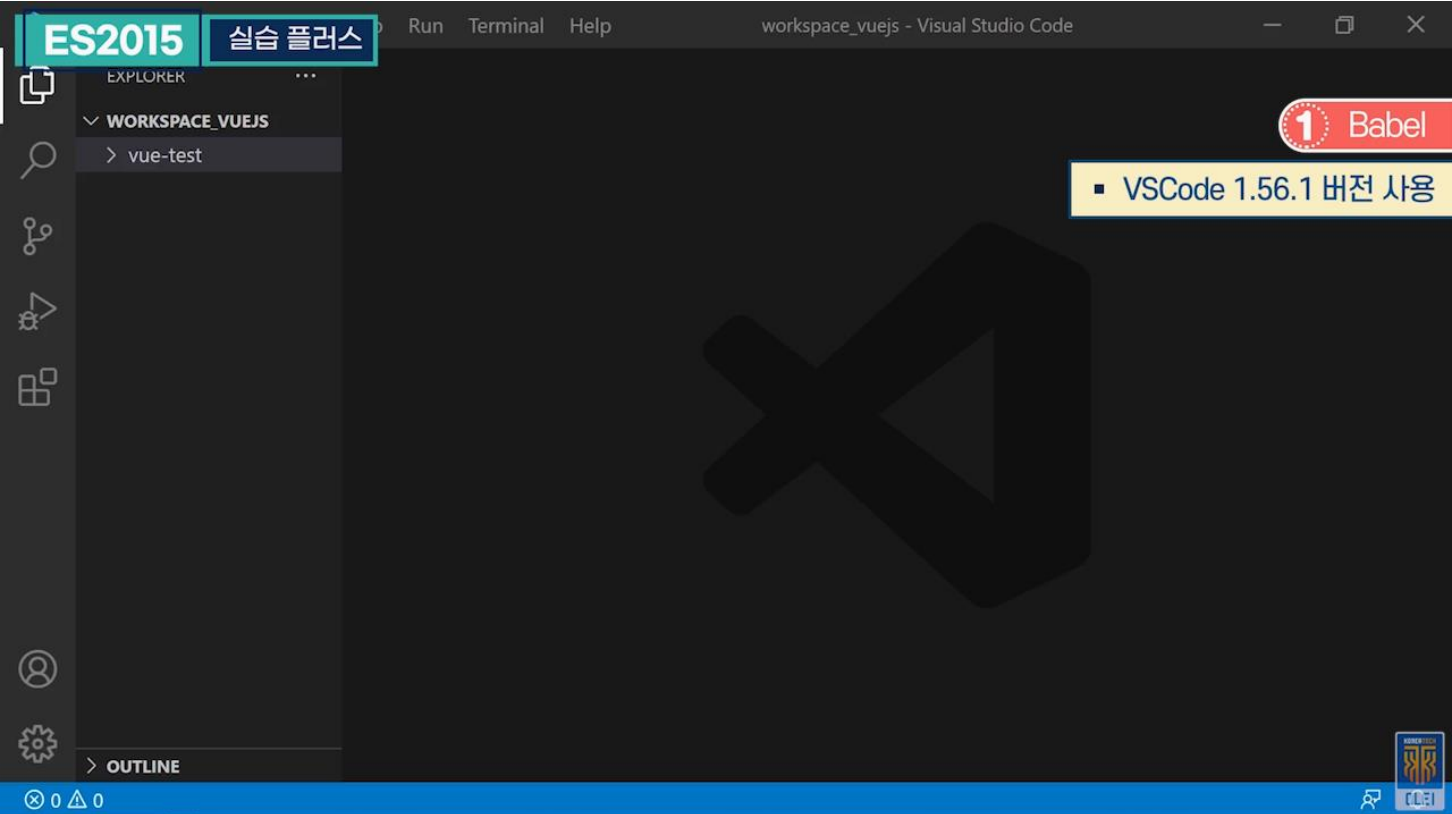
Babel

2 Babel

5 Compile 결과

```
(function () {
  function some(x) {
    var y = arguments.length > 1 && arguments[1] !== undefined ? arguments[1] : 10;
    var z = arguments.length > 2 && arguments[2] !== undefined ? arguments[2] : 20;
    console.log("x=".concat(x, ", y=").concat(y, ", z=").concat(z));
  }
  some(1); //x=1, y=10, z=20
  some(1, 2); //x=1, y=2, z=20
  some(1, 2, 3); //x=1, y=2, z=3
})();
```

실습 플러스



실습단계
ES2015와 Babel 이용 방법 테스트
ES2015를 테스트하기 위해 새로운 폴더를 만들어서 테스트
Vue 쪽에 프로젝트 포커스가 가지 않도록 새로운 폴더 만들기
ES2015 폴더 제작
ES2015 폴더를 누르고 서브 폴더 제작
ch3 폴더명으로 설정
ch3 폴더에 src 서브 폴더를 만들고 src 폴더 밑에 자바스크립트 파일 생성
src → main.js 자바스크립트 파일 생성
하나의 임의의 함수 정의, 함수가 선언되자마자 바로 호출한 경우라고 할 수 있음
some function 정의, 매개변수 세 개
두 번째, 세 번째 매개변수에 디폴트 값 설정
함수가 호출되었을 때 해당 값을 그대로 콘솔에 출력
매개변수 값을 그대로 콘솔에 출력한 코드
어디선가 some 함수를 Call 해주어야 함

실습 플러스

실습단계
매개변수 하나 주고 Call
매개변수 두 개를 주고 Call
매개변수 세 개를 주고 Call
함수 하나의 디폴트 값 명시, 싱글쿼트가 아니라 Tab 키 위의 backtick
문자열 내에 데이터를 바로 출력한 형태
자바스크립트 파일을 테스트하기 위해 HTML 파일 필요, 자바스크립트를 로딩하여 결과 확인
ch3 폴더에 HTML을 만들어서 테스트
ch3 폴더 → new file, HTML 파일 생성
파일명 : index.html
src → index.html, main.js
스크립트 로딩을 위해 스크립트 태그를 사용, 자바스크립트가 HTML에 의해 로딩되어 실행되게 처리
방금 만들어 놓은 main.js 지칭, src 밑의 main.js가 로딩되게 구성
HTML의 body 부분은 비워놓음
실행을 위해 서버가 필요함, HTML이 서버에서 동작해야 하고 브라우저로 테스트해줘야 함
Vue 프로젝트와 상관없이 ES2015를 살펴보기 위한 것
VS Code에 경량 서버를 설치하고 이용
Extensions → live 검색
쉽게 이용할 수 있는 경량 서버인 Live Server를 설치하고 이용할 수 있음
VS Code 하단 → Go Live, Live Server를 이용해서 HTML 로딩
1. HTML 페이지가 있는 상태에서 하단 Go Live 버튼 선택 2. HTML 페이지에서 오른쪽 마우스 버튼 눌러서 Open with Live Server 메뉴 이용
Live Server에서 HTML을 실행시키면 브라우저가 자동으로 뜸, 컴퓨터에 디폴트로 설정되어 있는 브라우저가 뜨게 됨
콘솔 출력 값을 확인하는 테스트
마우스 오른쪽 버튼 눌러서 콘솔창을 띄워야 함
작성한 자바스크립트 코드가 크롬 브라우저에서 정상적으로 수행됨
자바스크립트 코드가 모든 클라이언트의 브라우저에서 정상적으로 나온다고 볼 수 없음
어떤 브라우저에서 코드 문법이 맞지 않아 에러가 날 수 있음, 그 부분을 해결하기 위해 필요한 것이 Babel
작성했던 코드를 Babel로 컴파일 시켜서 서비스를 진행하는 형태

실습 플러스

실습단계
npm 명령으로 Babel 추가 설치가 필요함
설치하고자 하는 것을 나열하여 작성
Babel이 npm repository 서버로부터 다운로드 되어서 설치가 됨
Babel의 환경파일 명시 필요 → ch3 폴더에 Babel.config.js라는 이름의 파일을 다시 만듦
Babel의 환경파일 → Babel이 어떤 작업을 해야 하는지 명시함
js 파일을 Babel로 컴파일 시켜줘야 함
Babel 명령어를 이용하여 src의 파일을 컴파일 시켜 build 폴더에 컴파일 된 것을 명시할 것
build라는 폴더에 컴파일 된 결과를 출력
ES2015에서 작성된 코드가 모든 브라우저에서 수행된다고 보장할 수 없기 때문에 Babel을 이용하여 컴파일 시킨 후, 서비스하는 형태

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

ES2015 기초문법



한국기술교육대학교
온라인평생교육원

학습내용

- Constants & Template Literals
- Scoping
- Arrow Function

학습목표

- ES6의 **Constants와 Template Literals**에 대해 설명할 수 있다.
- ES6의 **Variable scope**에 대해 설명할 수 있다.
- ES6의 **Arrow Function**에 대해 설명할 수 있다.

Constants & Template Literals

Constants & Template Literals

1 Constants

① 상수변수

```
const PI = 3.141593
```

Constants & Template Literals

1 Constants

② 초기값 변경 불가

```
const a1 = "hello";  
a1 = 'world'; //-> 에러 (Uncaught TypeError: Assignment to constant variable.)
```

Constants & Template Literals

Constants & Template Literals

1 Constants

③ ES5 스타일 코드

```
(function () {
  var name = 'hello';
  console.log(name); //hello
  name='world'
  console.log(name); //world

  function someFun(){
    console.log("someFun 1.....")
  }
  someFun = function(){
    console.log("someFun 2....")
  }
  someFun()//someFun 2....
})();
```

Constants & Template Literals

1 Constants

④ ES6 스타일 코드

```
(function () {
  const name = 'hello';
  console.log(name);
  // name='world' //error
  console.log(name);

  const someFun = function(){
    console.log("someFun 1.....")
  }
  // someFun = function(){//error
  // console.log("someFun 2....")
  // }
  someFun()
})();
```


Constants & Template Literals

Constants & Template Literals

2 Template Literals

① 문자열에 동적 데이터 추가

→ 큰 따옴표(“ ”) 혹은 작은 따옴표(‘ ’)로 묶이는 문자열에 동적 데이터 추가는 + 를 이용하는 것이 기본

```
var name = "홍길동"
var message = "안녕하세요." + name + "님, 만나서 반갑습니다."
console.log(message)
```

Constants & Template Literals

2 Template Literals

② 문자열 내에 동적 데이터를 \${ } 내에 표현

→ 문자열이 backtick(` `) 으로 묶여 있어야 함

```
var name = "홍길동"
var message = `안녕하세요. ${name} 님, 만나서 반갑습니다.`
console.log(message)
```

Constants & Template Literals

Constants & Template Literals

2 Template Literals

3 실행 결과

안녕하세요. 홍길동 님, 만나서 반갑습니다.

Constants & Template Literals

2 Template Literals



Tip!

backtick(``) 으로 묶이는 문자열 데이터 내의
특수 키 값 유지

```
var order = { amount: 7, product: "Book", price: 100 }  
var order_message=  
`  
  product : ${order.product}  
  amount : ${order.amount}  
  price : ${order.price}  
`  
console.log(order_message)
```

Constants & Template Literals

Constants & Template Literals

2 Template Literals



Tip!

backtick(``) 으로 묶이는 문자열 데이터 내의
특수 키 값 유지

실행 결과

```
product : Book  
amount  : 7  
price   : 100
```

Scoping

Scoping

1 Scoping의 개념



변수 및 함수의 적용 범위

- ES5에서는 함수단위 Scoping만 지원함
- 블록 단위 Scoping은 지원하지 않음

```
if( )
{
}
}
```

```
for( )
{
}
}
```

File

```

□ → 변수/함수
Class A {
  ~□ 변수/함수
  a() {
    □
  }
}
}
```

Scoping

1 Scoping의 개념

ES5 스타일 코드

```

(function () {
  var name = 'hello'
  function a(){
    var name="world"
    console.log(name)//world
  }
  a()
  console.log(name)//hello
  if(true){
    var name="홍길동"
  }
  console.log(name)//홍길동
  {
    var name= " 김길동"
  }
  console.log(name)//김길동
})();
```

Scoping

Scoping

2 let 키워드

ES6에서는 변수 선언을 위한
let 키워드 제공

let 키워드로 선언된 변수는
블록단위 Scoping

Scoping

2 let 키워드

ES6 스타일 코드

```
(function () {
  let name = 'hello'
  function a(){
    let name="world"
    console.log(name)//world
  }
  a()
  console.log(name)//hello
  if(true){
    let name="홍길동"
  }
  console.log(name)//hello
  {
    let name= " 김길동"
  }
  console.log(name)//hello
})();
```

Arrow Function

Arrow Function

1 Arrow Function의 개념



화살표(=>)로 선언하는 함수

ES5 스타일 코드

```
evens.map(function (v) { return v + 1; });
evens.map(function (v) { return { even: v, odd: v + 1 }; });
evens.map(function (v, i) { return v + i; });
```

Arrow Function

1 Arrow Function의 개념



화살표(=>)로 선언하는 함수

ES6 스타일 코드

```
evens.map(v => v + 1)
evens.map(v => ({ even: v, odd: v + 1 }))
evens.map((v, i) => v + i)
```

Arrow Function

Arrow Function

2 Arrow Function 작성규칙

① 매개변수가 없는 Arrow Function

```
var fun1 = () => { console.log('i am fun1') }
```

Arrow Function

2 Arrow Function 작성규칙

② 매개변수를 가지는 Arrow Function

```
var fun2 = x => { console.log(`i am fun2, argument : ${x}`) }  
var fun3 = (x, y) => { console.log(`i am fun3, argument : ${x}, ${y}`) }
```

Arrow Function

Arrow Function

2 Arrow Function 작성규칙

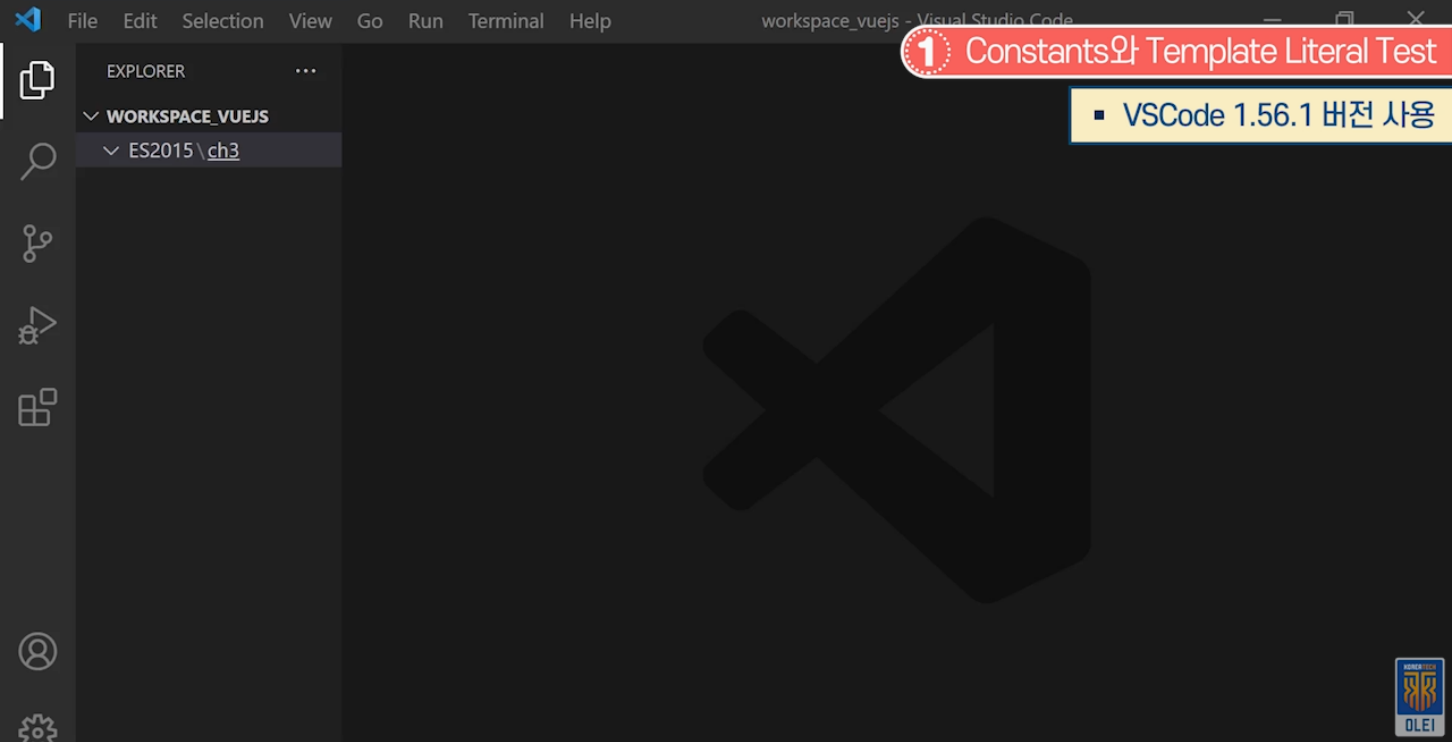
3 리턴 값이 있는 Arrow Function

```
var fun4 = x => { return x*x }  
var fun5 = x => x*x  
var fun6 = x => {  
  const y = x*x  
  return y  
}
```


실습 플러스

ES2015 기초문법

실습 플러스



실습단계
폴더 만들기 : ES2015 - ch4
ch4 선택 후 New File - main.js 입력
function을 만들고 호출해서 테스트
변수 선언, 값이 입력되어 있다는 가정
변숫값을 다른 문자열에서 포함시켜 출력해야 함
Message 변수를 만들어서 name 변수값을 뒤에 출력
문자열이 길고 동적데이터가 많을 경우 Template Literal 활용
문자열을 backtick(` `) 으로 표현
Backtick 내 문자열에 동적데이터 표현 부분을 \${} 로 표현
결과값 확인은 console.log(message) 출력, name 변수값 포함 여부 확인

실습 플러스

JS main.js

ES2015 > ch4 > JS main.js > <function>

2 Arrow Function Test

```

1  (function(){
2      var name = "홍길동"
3      var message = `안녕하세요... ${name}`
4      console.log(message)
5
6
7  })()
```



실습단계

간단한 함수는 화살표(=>) 함수로 표현

fun1 변수에 화살표 함수 대입, 함수도 데이터로 취급, 변수에 대입

화살표(=>)를 기준으로 왼쪽이 매개변수, '()'는 매개변수가 없는 함수

함수가 정상적으로 Call 되었는지 확인을 위해 console.log("I am fun1....") 입력

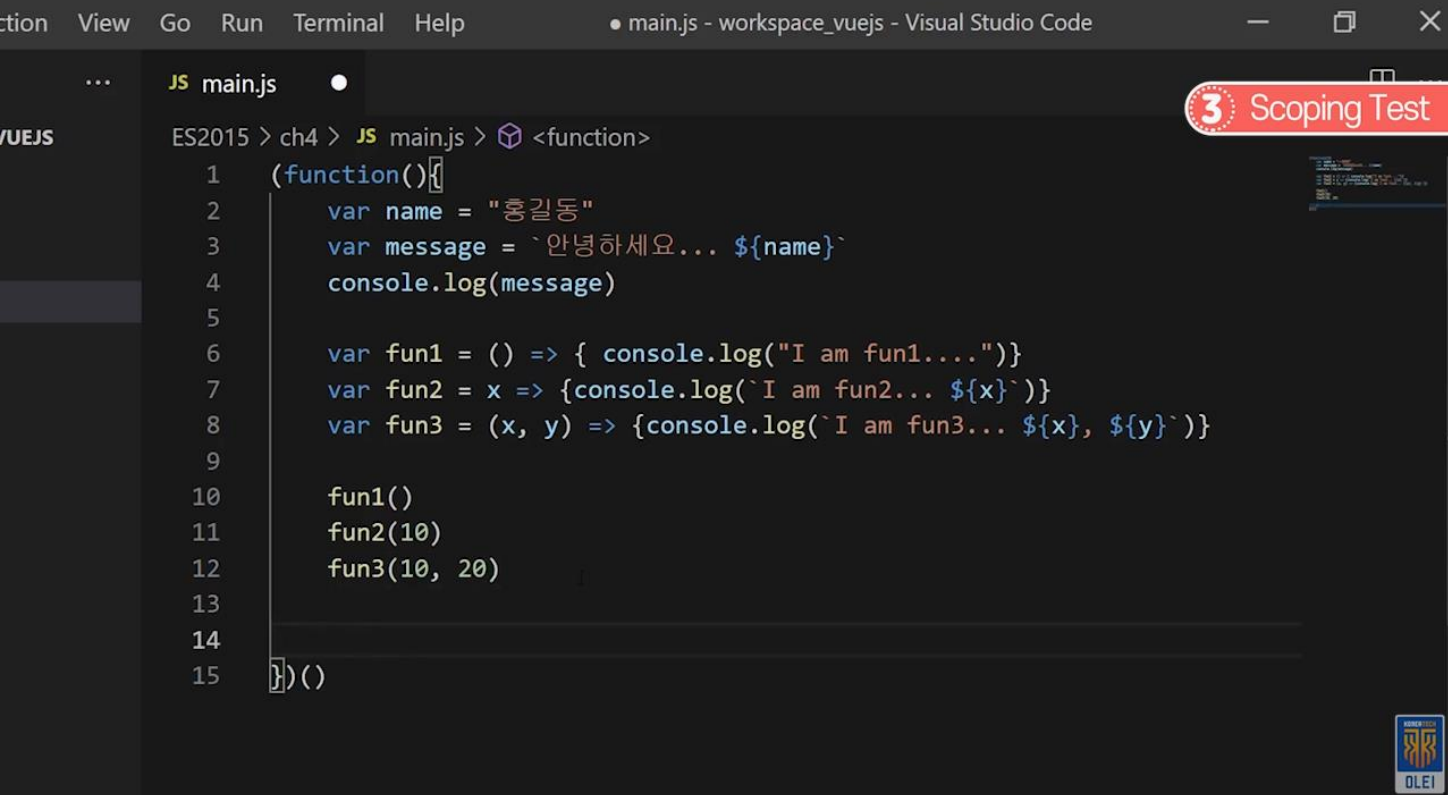
매개변수를 지정하여 다른 함수 선언

매개변수 x(매개변수를 x로 핸들링)
console.log('I am fun2... \${x}')

매개변수가 여러 개일 경우
(x, y) => {console.log('I am fun3... \${x}, \${y}')}

함수 호출 및 매개변수 값 전달

실습 플러스



실습단계
변수 선언을 var로 하는 대신, let으로 선언
테스트를 위해 if문 작성
* Scoping 안 되었을 경우 Outer 변수명과 if문 안의 변수명이 동일하여 Outer 변수값이 "world" 로 바뀜
* Scoping 될 경우 영역이 달라 if문 내에서만 "world" 값이 유지됨
console.log(data)로 무슨 값이 나오는지 확인
HTML을 만들어야 자바스크립트 테스트 가능
ch4 - New File - index.html
"main.js"를 단순하게 실행
Open with Live Server 클릭
빈 화면에서 오른쪽 마우스 클릭, 검사 - console 작성한 내용 확인

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

ES2015 – OOP



한국기술교육대학교
온라인평생교육원

학습내용

- Class와 생성자
- Property

학습목표

- Class 선언과 생성자, 상속에 대해 설명할 수 있다.
- Property 선언과 setter, getter 이용 방법에 대해 설명할 수 있다.

Class와 생성자

Class와 생성자

1 Class 선언

- ES5에서는 함수로 클래스 정의함

```
var User = (function(){
  function User(name){
    this.name=name
  }
  User.prototype.sayHello = function(){
    console.log(`sayHello...${this.name}`)
  }
  return User
})();
```

Class와 생성자

1 Class 선언

- ES5에서는 함수로 클래스 정의함

```
var obj1=new User('kkang')
var obj2=new User('kim')
obj1.sayHello()//sayHello...kkang
obj2.sayHello()//sayHello...kim
```

Class와 생성자

Class와 생성자

1 Class 선언

- ES2015에서는 class 예약어로 클래스 선언함

```
class User {
  constructor(name){
    this.name=name
  }
  sayHello(){
    console.log(`sayHello...${this.name}`)
  }
}
```

Class와 생성자

1 Class 선언

- ES2015에서는 class 예약어로 클래스 선언함

```
var obj1=new User('kkang')
var obj2=new User('kim')
obj1.sayHello()//sayHello...kkang
obj2.sayHello()//sayHello...kim
```

Class와 생성자

Class와 생성자

2 생성자

생성자

- 객체 생성 시점에 호출되는 함수
- constructor 이름으로 선언되는 함수
- 하나의 클래스 내에 하나의 constructor 함수만 선언 가능함

Class와 생성자

2 생성자

- 생략 가능하며 생략하면 매개변수 없는 constructor가 자동 추가됨

```
class User{}  
  
class User1{  
  constructor(){}  
}
```


Class와 생성자

Class와 생성자

2 생성자

- 생략 가능하며 생략하면 매개변수 없는 constructor가 자동 추가됨

```
class User2{
  constructor(name){}
}

var obj1=new User()
var obj2=new User1()
var obj3=new User2('kkang')
```

Class와 생성자

3 상속

- extends 예약어로 상속 관계 명시
- super 예약어로 상위 클래스의 생성자, 멤버 지칭

```
class Shape {
  constructor(id, x, y) {
  }
}
class Rectangle extends Shape {
  constructor(id, x, y, width, height) {
    super(id, x, y)
  }
}
var obj = new Rectangle(1, 0, 0, 100, 100)
```

Property

Property

1 Property

Property

- 클래스의 멤버 변수
- 클래스 선언영역에서 선언

Property

1 Property

- 클래스 내에서 멤버 변수 접근은 this 예약어를 이용함

```
class User {  
  name  
  address='seoul'  
  constructor(arg1, arg2){  
    this.name = arg1  
  }  
  sayHello(){  
    console.log(`${this.name}, ${this.address}`)//hello, seoul  
  }  
}
```

Property

Property

1 Property

- 클래스 내에서 멤버 변수 접근은 this 예약어를 이용함

```
var user = new User("hello", "a@a.com")
user.sayHello()
```

Property

1 Property

- 클래스 선언영역이 아닌 생성자 내부 혹은 함수 내부에서도 클래스 멤버 변수 선언이 가능함

```
class User {
  constructor(arg1, arg2){
    this.name = arg1
    this.email = arg2
  }
  someFun() {
    this.age = 30
  }
  sayHello(){
    console.log(`${this.name}, ${this.email}, ${this.age}`)
    //hello, a@a.com, 30
  }
}
```

Property

Property

1 Property

- 클래스 선언영역이 아닌 생성자 내부 혹은 함수 내부에서도 클래스 멤버 변수 선언이 가능함

```
var user = new User("hello", "a@a.com")
user.someFun()
user.sayHello()
```

Property

2 Getter/Setter

- Getter, Setter는 get과 set 예약어로 선언되는 함수
- 외부에서는 Property로 이용

```
class Product {
  constructor(arg){
    this._name = arg
  }
  get name() { return this._name }
  set name(arg) { this._name = arg }

  get price() { return this._price }
  set price(arg) { return this._price }
}
```

Property

Property

2 Getter/Setter

- Getter, Setter는 get과 set 예약어로 선언되는 함수
- 외부에서는 Property로 이용

```
var prod = new Product('book1')
prod.name = 'book2'
prod.price = 200

console.log(`${prod.name}, ${prod.price}`)
```

Property

2 Getter/Setter

get 예약어로
선언되는 함수

매개변수가 **없어야 함**

set 예약어로
선언되는 함수

매개변수가 **하나**만 선언되어야 함

Property

Property

2 Getter/Setter

- set, get 선언되는 함수를 외부에서 함수로 이용할 수는 없음

```
class Product {
  get price() { return this._price }
  set price(arg) { this._price = arg }

  get code(arg) { return this._code } //error
  set code(arg1, arg2) { this._code = arg1 } //error
}

var prod = new Product('book1')
prod.price('100') //error
prod.price = 200
```

Property

3 static 함수

static 함수

- static 예약어로 추가되는 함수
- 클래스 명으로 접근해야 함(객체로 접근 불가)

Property

Property

3 static 함수

- static 함수에서 Object member 접근하면 Undefined임

```
class User2 {
  data = 10
  static fun1() {
    console.log(`data : ${this.data}`)//undefined
  }
  fun2() {
    console.log(`data : ${this.data}`)
  }
}
```

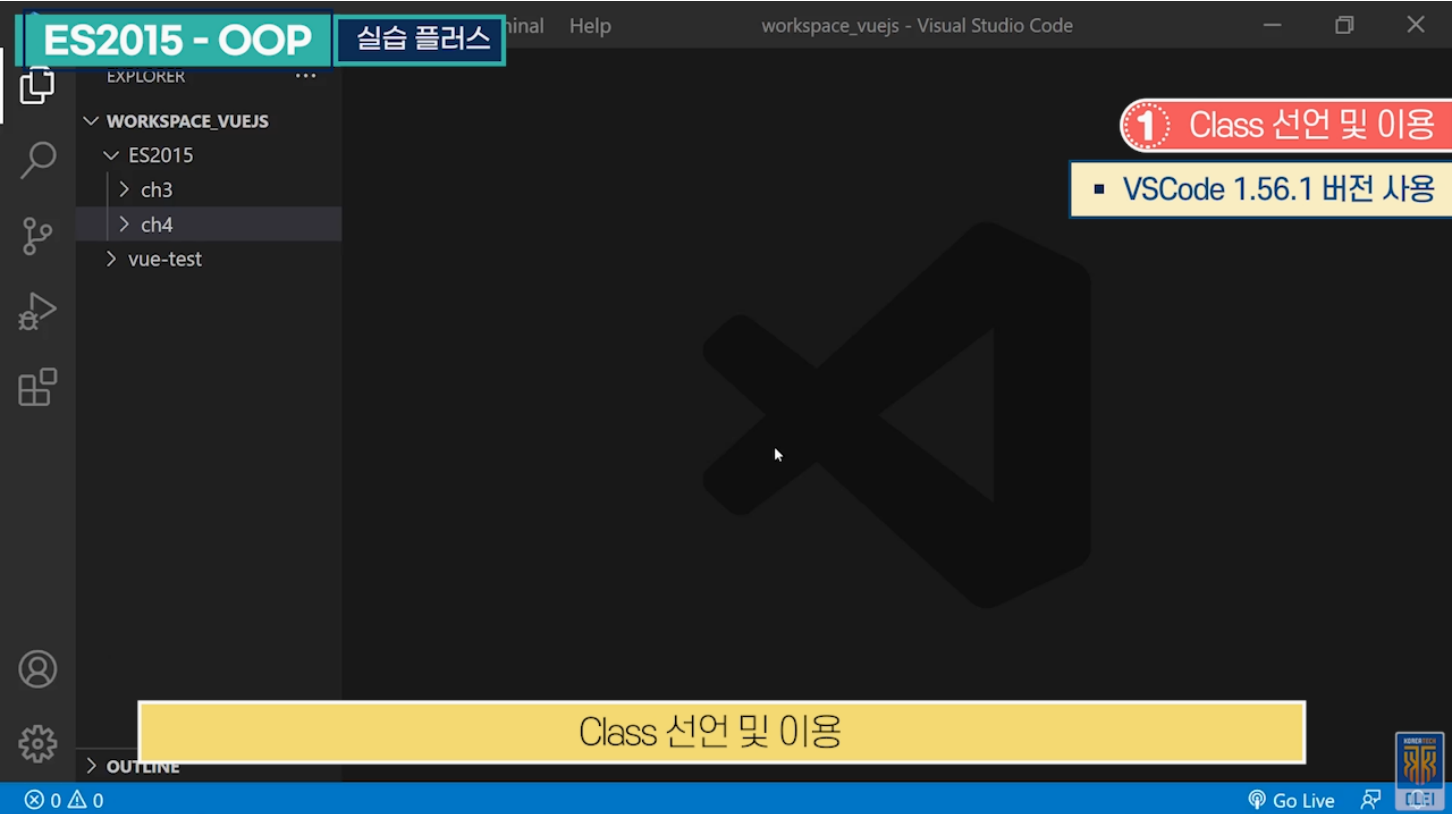
Property

3 static 함수

- static 함수에서 Object member 접근하면 Undefined임

```
User2.fun1()
// User2.fun2()//error
var user2 = new User2()
// user2.fun1()//error
user2.fun2()
```

실습 플러스



실습단계
ES2015 → ch5 서브 폴더 생성
서브 폴더 생성 후 자바스크립트 파일 생성
Customer Class 내에 멤버변수 name, address 선언
매개변수 두 개를 받는 constructor 선언
받은 매개변수로 멤버변수 초기화
멤버변수를 Class 영역에서 선언하지 않아도 됨
someFun 함수 선언
일반 함수에서도 멤버변수 선언 가능
console 창에 멤버변수를 출력하는 함수 선언
Getter/Setter 정의
객체 생성, 매개변수 전달
생성한 객체의 함수 호출
테스트용 index.html 파일 생성
Script Tag로 자바스크립트 파일을 지정함

실습 플러스

실습단계
Live 서버로 실행
console에서 데이터 확인 가능
Class 상속 테스트
ch5 서브 폴더에 inheritance.js 파일 생성 후 Class 선언
생성자를 만들고 멤버변수 초기화
함수 선언, console에 출력 정의
만들어진 Class를 상속받아 다른 Class 생성
상위 Class 생성자 호출
상위 Class 함수 오버라이드
Class 이용을 위해 객체 생성
방금 작성한 자바스크립트 파일을 추가함

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

ES2015 - 다양한 기법



한국기술교육대학교
온라인평생교육원

학습내용

- Argument, Parameter, Operation
- Destructured, Object literal, Modules

학습목표

- Default argument, Rest parameter, Spread operation에 대해 설명할 수 있다.
- Destructured assignment, Object literal, Modules 이 용에 대해 설명할 수 있다.

Argument, Parameter, Operation

Argument, Parameter, Operation

1 Default Argument

- 함수의 매개변수 선언할 때 기본 값 지정

```
function some(x, y = 10, z = 20) {  
  console.log(`x=${x}, y=${y}, z=${z}`)  
}  
some(1)//x=1, y=10, z=20  
some(1, 2)//x=1, y=2, z=20  
some(1, 2, 3)//x=1, y=2, z=3
```

Argument, Parameter, Operation

2 Spread operator

- 배열, 객체를 각 요소로 분리시키는 연산자
- 배열 복사, 객체 결합 등에 유용하게 사용됨

Argument, Parameter, Operation

Argument, Parameter, Operation

2 Spread operator

배열 복사

```
var array1=[10,20,30]
var array2 = [1, 2, ...array1]
console.log(array2)//[1, 2, 10, 20, 30]

array2.push(...array1)
console.log(array2)//[1, 2, 10, 20, 30, 10, 20, 30]
```

Argument, Parameter, Operation

2 Spread operator

Set 객체 복사

```
var datas = new Set([1,1,3,4])
var array3 = [10, 20, ...datas]
console.log(array3)//[10, 20, 1, 3, 4]
```

Argument, Parameter, Operation

Argument, Parameter, Operation

2 Spread operator

배열 복사

```
var obj = {  
  data1: 'hello',  
  data2: 10  
}  
var obj2 = {  
  ...obj,  
  data3: true  
}  
  
console.log(obj2)//{data1: "hello", data2: 10, data3: true}
```

Argument, Parameter, Operation

3 Rest Parameter

- 함수의 Argument를 Spread operator(...)을 이용하여 선언하는 기법
- Rest parameter 값은 함수 내에서 배열로 이용

Argument, Parameter, Operation

Argument, Parameter, Operation

3 Rest Parameter

- 함수의 마지막 Argument만 Rest parameter로 선언이 가능

```
function some (x, y, ...z) {  
  console.log(z.length)//3  
  z.forEach( arg => {  
    console.log(arg)//hello - true - 2  
  })  
}  
some(1, 2, "hello", true, 2)
```

Destructured, Object literal, Modules

Destructured, Object literal, Modules

1 Destructured assignment

- 객체 혹은 배열을 분해 하는 작업
- 변수 명 변경 혹은 default 값 명시 가능

Destructured, Object literal, Modules

1 Destructured assignment

- Destructured assignment를 사용하지 않은 경우의 예

```
var obj = {  
  data1: 'hello',  
  data2: 10,  
  data3: true  
}
```

```
let data1 = obj.data1  
let data2 = obj.data2  
let data3 = obj.data3
```


Destructured, Object literal, Modules

Destructured, Object literal, Modules

1 Destructured assignment

● Destructured assignment 사용 예

```
var obj = {  
  data1: 'hello',  
  data2: 10,  
  data3: true  
}  
  
let {data1, data2, data3} =  
obj
```

Destructured, Object literal, Modules

1 Destructured assignment

● Default 값 명시

```
var obj = {  
  data1: 'hello',  
  data2: 10,  
  data3: true  
}  
  
let {data1, data2 = 20, data3, data4 = 'world'} = obj
```

Destructured, Object literal, Modules

Destructured, Object literal, Modules

1 Destructured assignment

- 이름 변경 획득

```
var obj = {
  data1: 'hello',
  data2: 10,
  data3: true
}
```

```
let {data1: a1, data2: a2 = 20, data3: a3, data4: a4 = 'world'} = obj
```

Destructured, Object literal, Modules

1 Destructured assignment

- 배열 분해도 제공
- 배열 분해는 중괄호({})를 사용하지 않고 대괄호([])를 사용함

```
var array = ["hello", "world"]
let [b1, b2, b3 = true] = array
```

Destructured, Object literal, Modules

Destructured, Object literal, Modules

2 JSON

- Key - value 쌍으로 표현하는 데이터 표현 방식

```
var json = {
  "data1": "hello",
  "data2": 10,
  "data3": true
}
```

Destructured, Object literal, Modules

3 Object Literal

JSON	특징
JavaScript Object Notation	Key: Value 쌍으로 표현하는 객체 표현 방식

```
var user = {
  name: 'hong',
  email: 'a@a.com',
  age: 30
}

var name = user.name
```

Destructed, Object literal, Modules

Destructed, Object literal, Modules

3 Object Literal

- 객체 표현임으로 함수 등록 가능

```
var user = {  
  name: 'hong',  
  email: 'a@a.com',  
  age: 30,  
  sayHello: function(arg){  
    console.log(`Hello.. ${arg}`)  
  }  
}
```

Destructed, Object literal, Modules

3 Object Literal

- Object 구성할 때 축약형으로 구성 가능

```
var name = 'hong'  
var email = 'a@a.com'  
  
var user = {  
  name: name,  
  email,  
  age: 30,  
}
```

Destructed, Object literal, Modules

Destructed, Object literal, Modules

3 Object Literal

- 객체 내의 멤버는 this 예약어로 이용

```
var user = {  
  name: name,  
  email,  
  age: 30,  
  sayHello: function(){  
    console.log(`Hello.. ${this.name}, ${this.email}, ${this.age}`)  
  }  
}
```

Destructed, Object literal, Modules

3 Object Literal

- JSON 문자열 파싱
- JSON 문자열을 Object로 파싱해서 사용

Destructed, Object literal, Modules

Destructed, Object literal, Modules

3 Object Literal

- JSON.parse() 이용

```
const json = '{"result":true, "count":42}'  
console.log(json.result)//undefined
```

```
const json = '{"result":true, "count":42}'  
const obj = JSON.parse(json)  
console.log(obj.result)//true
```

Destructed, Object literal, Modules

3 Object Literal

- Object Literal을 JSON 문자열로 변환
- JSON.stringify() 이용

Destructed, Object literal, Modules

Destructed, Object literal, Modules

3 Object Literal

```
var user = {
  name: name,
  email,
  age: 30,
  sayHello: function(){
    console.log(`Hello.. ${this.name}, ${this.email}, ${this.age}`)
  }
}

const str = JSON.stringify(user)
console.log(str)
console.log(user)
```

Destructed, Object literal, Modules

3 Object Literal

```
{ "name": "hong", "email": "a@a.com", "age": 30 }
▼ {name: "hong", email: "a@a.com", age: 30, sayHello: f} ⓘ
  age: 30
  email: "a@a.com"
  name: "hong"
  ▶ sayHello: f ()
  ▶ [[Prototype]]: Object
```

Destructured, Object literal, Modules

Destructured, Object literal, Modules

4 Modules

- 자바스크립트 파일의 구성요소(변수, 함수, 클래스)를 외부 파일에 공개 및 이용

```
// a.js
export function sum (x, y) { return x + y }
export var pi = 3.141593

// b.js
import * as math from "a"
console.log("2π = " + math.sum(math.pi, math.pi))

// c.js
import { sum, pi } from "a"
console.log("2π = " + sum(pi, pi))
```

Destructured, Object literal, Modules

4 Modules

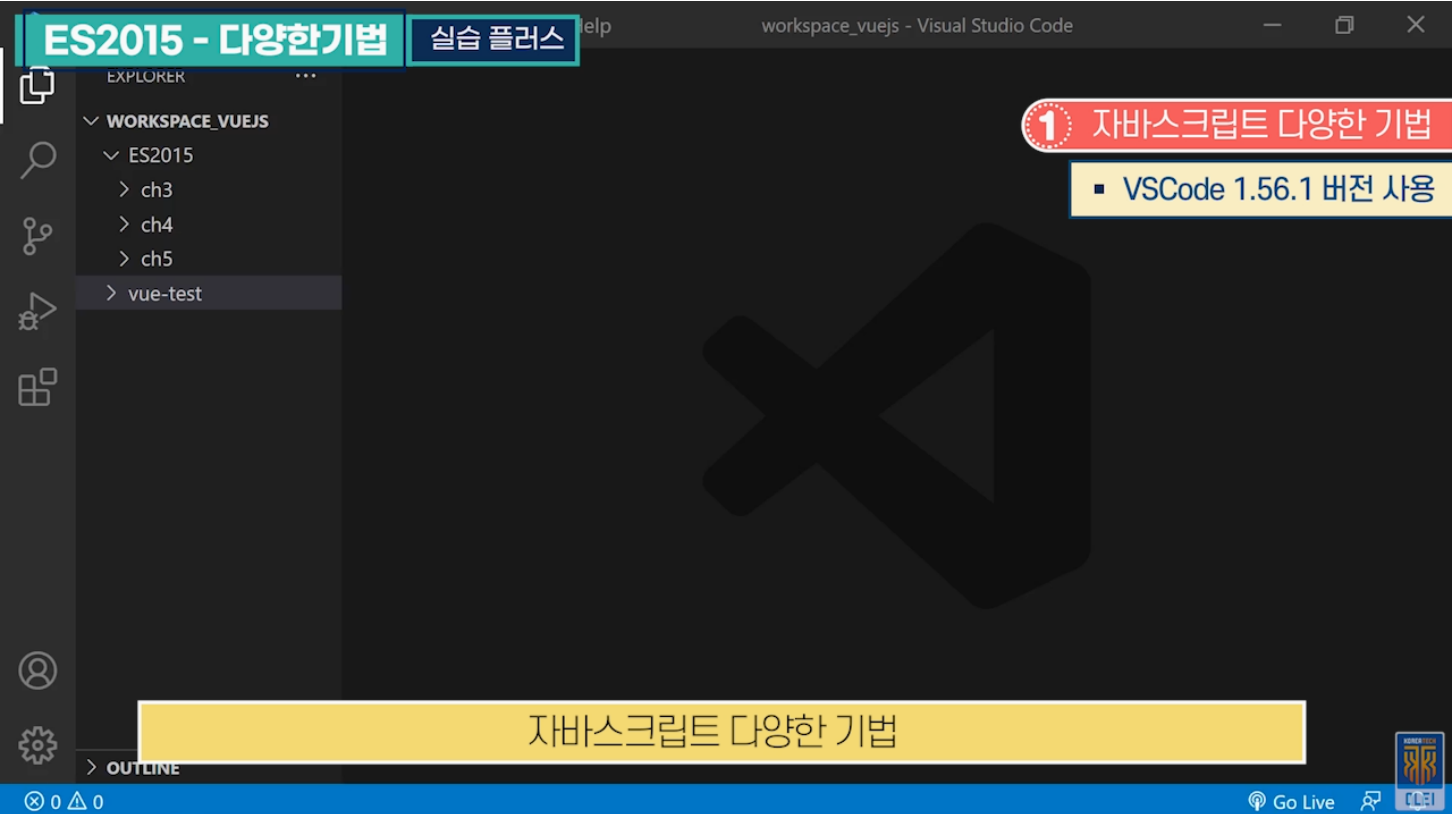
- import 방법

1 import myModule from 'my-module';

2 import 'my-module' as myModule;

3 var myModule = require('my-module');

실습 플러스



실습단계
실습을 위해 임의의 폴더를 하나 만들어서 테스트, 폴더명 ch6
New File → 자바스크립트 파일 제작, 파일명 main.js
테스트를 구분하기 위해 console 사용
제일 먼저 스프레드 연산자(Spread operator) 테스트
임의의 배열이 선언됨, 배열을 복제하여 다른 배열을 만들겠다는 가정
array2라는 배열 선언, array1의 모든 배열 데이터를 복제하겠다는 의미
원래 배열 데이터를 하나하나 추출해서 담으면 됨, 배열데이터가 많으면 스프레드 연산자로 한번에 복제 가능
스프레드 연산자로 Object literal의 구성 요소 복제 가능
스프레드 연산자로 한번에 복제 가능
Rest Parameter 테스트, 함수의 매개변수 부분
함수의 매개변수를 정확하게 지정하는 것이 좋음
함수 선언하면서 매개변수 개수가 많거나 예측하기 힘들 때 Rest parameter를 사용하면 유용함
함수를 호출했을 때, 세 번째 매개변수에 여러 개의 값이 들어옴

실습 플러스

실습단계
각각의 값이 대입되었는지 console로 확인
some 함수를 호출하면 x, y에는 두 개의 데이터를 대입하고 나머지 매개변수는 Rest Parameter에 대입
호출하면, 세 개의 값은 Rest Parameter로 되어 있는 매개변수에 대입하게 됨
Destructured assignment 테스트, Object literal이 필요함
여러 개를 추출할 때 유용하게 사용할 수 있는 기법
obj3를 대입하면, obj3의 데이터 중에서 data1 값을 추출하여 a1이라는 변수에 담으라는 것
데이터를 추출하여 a2에 담고, 데이터가 없을 경우 디폴트 값은 20이라는 선언
제대로 데이터가 추출돼서 변수에 들어갔는지 log로 확인
테스트하기 위해 HTML 파일 하나가 필요함, ch6 → 파일명 index.html 생성
HTML을 Live Server로 테스트 진행
검사를 눌러서 console로 확인
Array, Object literal 데이터, Rest Parameter 값, Destructured assignment 데이터 추출 결과 값 확인

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Component 기본



한국기술교육대학교
온라인평생교육원

학습내용

- Component 선언
- Component 이용

학습목표

- Component를 선언하는 방법에 대해 설명할 수 있다.
- Component의 Global 등록 및 Local 등록 방법에 대해 설명할 수 있다.

Component 선언

Component 선언

1 Component



Component 특징

- Vue의 핵심 구성요소
- UI를 위한 화면(DOM)과 로직(Javascript) 결합



Component



The Progressive
JavaScript Framework

WHY VUE.JS?

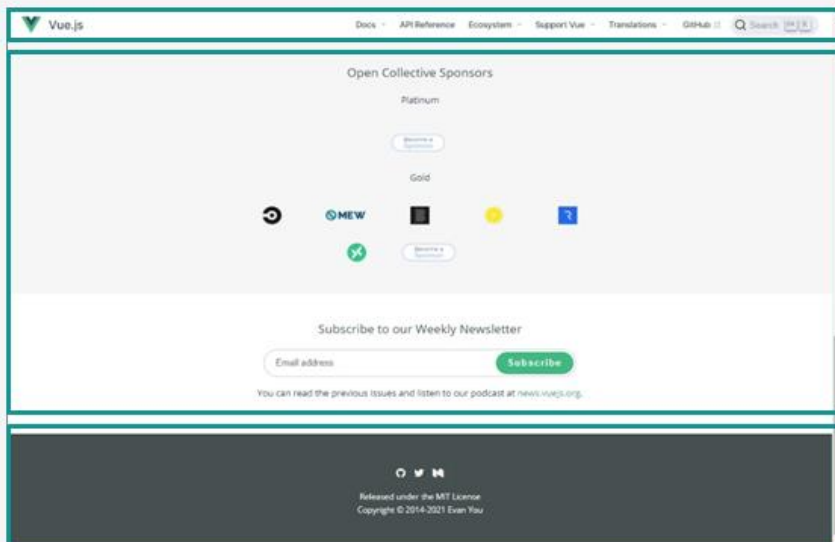
GET STARTED

GITHUB

Component 선언

1 Component

- 재사용 가능한 동적 UI 구성을 목적으로 함



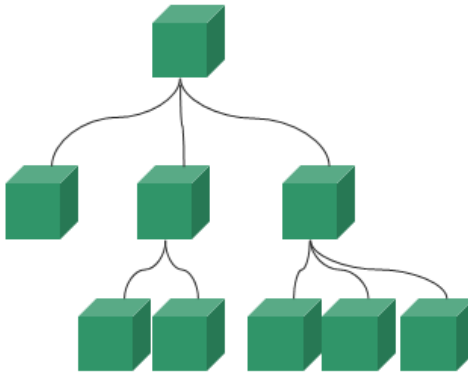
Component 선언

Component 선언

1 Component

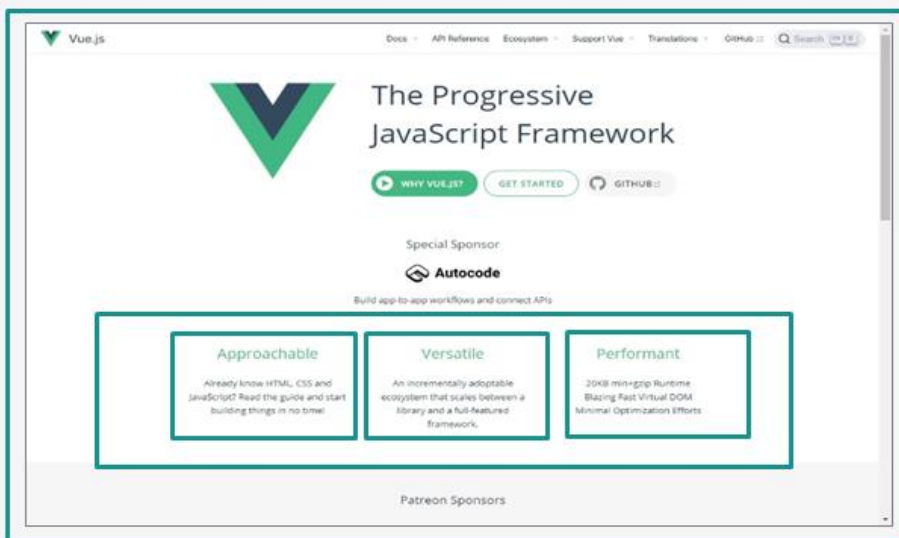
Component Tree 구조

→ 의미 단위로 Component를 구성하고
구성된 Component를 조합해서 화면을 구성함



Component 선언

1 Component



Component 선언

Component 선언

2 Object Literal로 Component 선언

- Template, Method, Data 등의 다양한 Option을 가지는 **Object literal**로 선언함
- Component의 Data는 **Data(){} 스타일**의 함수로 선언
- Component의 화면은 **Template 속성**으로 정의함

Component 선언

2 Object Literal로 Component 선언

- Component의 Data를 Template에서 이용할 때는 **{{ }} 표현식** 이용

```
var myComponent = {
  data() {
    return {
      name: 'Hong Gildong'
    }
  },
  template: `
    <h3>Component Test {{name}}
  `
}
```

Component 선언

Component 선언

3 Application Instance를 이용해 Component 출력

- createApp() 함수로 Application Instance 생성
- 매개변수에 Root Component 등록

Component 선언

3 Application Instance를 이용해 Component 출력

- mount() 함수로 화면 출력

```
var app = createApp(myComponent)
app.mount('#app')
```


Component 선언

Component 선언

4 Vue 파일로 Component 선언

- 확장자가 Vue인 파일

<template>, <script>, <style> 태그로 Component 구성

Component 선언

4 Vue 파일로 Component 선언

```
<template>
  <div>
    Hello,
    {{ name }}
  </div>
</template>
<script>
export default {
  data(){
    return {
      name: ' vue...'
    }
  }
};
</script>
```

Component 이용

Component 이용

1 Global 등록 사용



Application Instance에 Component 등록



Application 전역에서 Component 이용



Application Instance의 component()함수로 등록

Component 이용

1 Global 등록 사용

- 첫 번째 매개변수가 Component 태그명임

```
var app = createApp(Super)
app.component('Sub', Sub)
```

Component 이용

Component 이용

1 Global 등록 사용

● 등록된 Component 이용

```
<template>
  <div>
    I am super component<br/>
    <Sub/>
  </div>
</template>
<script>
export default {
};
</script>
```

Component 이용

2 Local 등록 사용



Component가 필요한 곳에서 등록 사용



components 속성을 이용해 등록

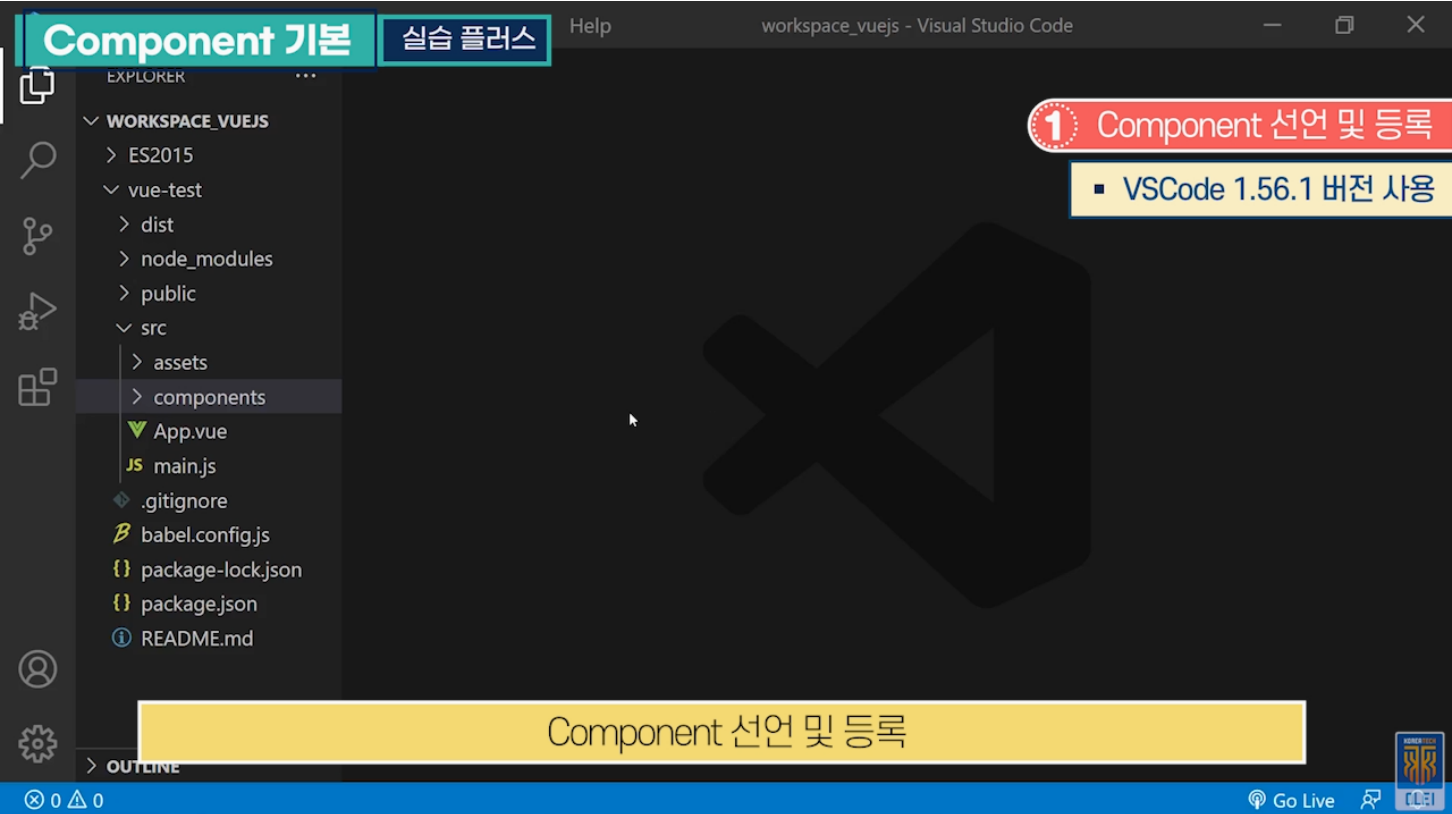
Component 이용

Component 이용

2 Local 등록 사용

```
<template>
  <div>
    I am super component<br/>
    <MySub/>
  </div>
</template>
<script>
import MySub from "./Sub.vue";
export default {
  components: {
    MySub
  }
};
</script>
```

실습 플러스



실습단계
Vue 프로젝트에 Component 생성 실습
src → ch7 서브 폴더 생성
파일명을 test1.js로 해서 자바스크립트 파일 생성
Component 정의
변수 값을 출력할 템플릿 정의
외부 파일에서 사용이 가능하도록 만들어진 Component를 export
해당 Component를 Vue 프로젝트에 적용시키기 위해 main.js 파일 생성
Vue 라이브러리와 작성한 js 파일을 import
Vue의 App 객체를 create
프로젝트를 화면에 출력하기 위한 선언
동일 폴더에 HTML 파일 생성, index.html
body 부분에 프로젝트 출력에 필요한 Tag만 명시
테스트 편의성, 개발자 임의의 폴더만 별도 테스트 진행
ch7 폴더에 Vue 설정을 위한 js 파일 생성, vue.config.js

실습 플러스

실습단계
ch7 폴더만 독립적으로 Vue CLI 명령으로 테스트 진행 → 테스트 편의성
Npm 명령어로 테스트를 위한 모듈 설치
Vue CLI 명령으로 실행
브라우저에 Component 출력
Component를 자바스크립트로 정의함
효율적인 방법은 확장자가 vue인 파일로 Component를 정의하는 것!
VS Code가 확장자가 vue인 파일을 인지하지 못함!
VS Code 플러그인 설치, Extensions 클릭
vetur 검색
Vue를 VS Code에서 인지할 수 있도록 플러그인 vetur 설치
Vue 파일 생성 가능!
Vue 파일에 Component 정의
스크립트 Tag 영역에 Vue Component 정의
변수 값이 템플릿에 출력되어서 템플릿 내의 내용이 화면에 출력되는 것
main.js 파일에 적용
저장을 하면 Hot Deploy에 의해 결과가 바로 출력됨
브라우저에 Vue 파일로 작성한 Component 출력

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Component Option과 Data Binding



한국기술교육대학교
온라인평생교육원

학습내용

- Component Option
- Data Binding

학습목표

- **Component Option**을 이해하고 사용하는 방법에 대해 설명할 수 있다.
- **Component template**에 data를 바인딩할 수 있다.

Component Option

Component Option

1 components

- Child Component를 등록하기 위한 옵션

```
components: {  
  MySub  
}
```

Component Option

2 template

template

- Component의 UI Template
- 데이터가 바인딩 되어야 하는 부분을 {{ }}으로 표현

Component Option

Component Option

2 template

- Component의 데이터가 변경되면 Template에 의해 출력되는 부분도 변경

```
template: `
  <div>
    <div>message : {{ msg }}</div>
  </div>
`
```

Component Option

3 data

data

- Component를 위한 data 설정
- data에 설정된 값을 template, method 등에서 이용

Component Option

Component Option

3 data

- data 옵션은 함수 형식으로 선언

```
data() {
  return {
    msg: 'hello',
    arrayData: [
      { message: 'hello' },
      { message: 'world' }
    ]
  }
},
```

Component Option

4 methods

- Component에 메서드를 등록하기 위한 옵션

```
<p>Reversed message: "{{ reversedMessage() }}"</p>
.....
methods: {
  reversedMessage: function () {
    return this.msg.split('').reverse().join('')
  }
}
```

Component Option

Component Option

5 computed

computed

- 템플릿 표현식의 복잡함을 줄이기 위해서 제공
- 표현식을 함수로 정의한 후 computed 속성에 등록하면 템플릿에서 Getter Property로 이용

Component Option

5 computed

computed

- 캐싱 제공

methods

- 캐싱 기능이 없음

Component Option

Component Option

5 computed

- 값이 바뀌지 않으면 반복 호출되지 않음

```
<p>Computed reversed message: "{{ reversedMessage }}"</p>
.....
computed: {
  reversedMessage: function () {
    return this.msg.split('').reverse().join('')
  }
}
```

Component Option

5 computed

- 기본으로는 Getter 함수만 제공이 되지만 원한다면 Setter 함수 추가 가능
- Setter 함수도 Property로 이용함
fullName='kim'

```
fullName: {
  get: function () {
    .....
  },
  set: function (newValue) {
    .....
  }
}
```

Component Option

Component Option

6 watch

🕒 Data 변경 감지

```
watch: {  
  firstName: function (val) {  
    this.fullName = val + ' ' + this.lastName  
  },  
  lastName: function (val) {  
    this.fullName = this.firstName + ' ' + val  
  }  
}
```

Data Binding

Data Binding

1 Component Data Binding

- Component Data를 화면에 출력함
- Data는 `data() {}`에 명시
- Template에서 Data Binding은 `{{ }}` 표현식 이용

Data Binding

1 Component Data Binding

```
<template>
  <div>
    Hello,
    {{ name }}
  </div>
</template>
<script>
export default {
  data(){
    return {
      name: ' vue...'
    }
  }
};
</script>
```

Data Binding

Data Binding

2 Tag Attribute 값에 Data Binding

- HTML 속성 값에 데이터 바인딩은 **{{ }}**을 사용할 수 없음
- v-bind**를 이용해 속성 값을 지정함

```
<div v-bind:id="dynamicId"></div>
.....
data(){
  return {
    dynamicId: 'a1'
  }
}
```

Data Binding

2 Tag Attribute 값에 Data Binding

- 값이 Boolean이 대입되는 속성 경우 데이터 값이 **false**이며, **undefined, null**이면 속성이 추가되지 않음

```
<button v-
bind:disabled="isButtonDisabled">Button</button>
.....
data(){
  return {
    isButtonDisabled: true
  }
}
```


Data Binding

Data Binding

3 Data Binding 관련 Directive

v-once

초기 데이터 출력 후 데이터 변경에도 업데이트 되지 않는 Template

```
<span v-once>This will never change:
{{ msg }}</span>
```

v-html

rawHtml로 출력

```
<p>Using mustaches: {{ rawHtml }}</p>
<p>Using v-html directive:
<span v-html="rawHtml"></span></p>
```

Data Binding

4 표현식 출력

- Template의 {{ }} 혹은 v-bind 안에 자바스크립트 표현식이 가능
- 단일 표현식만 가능

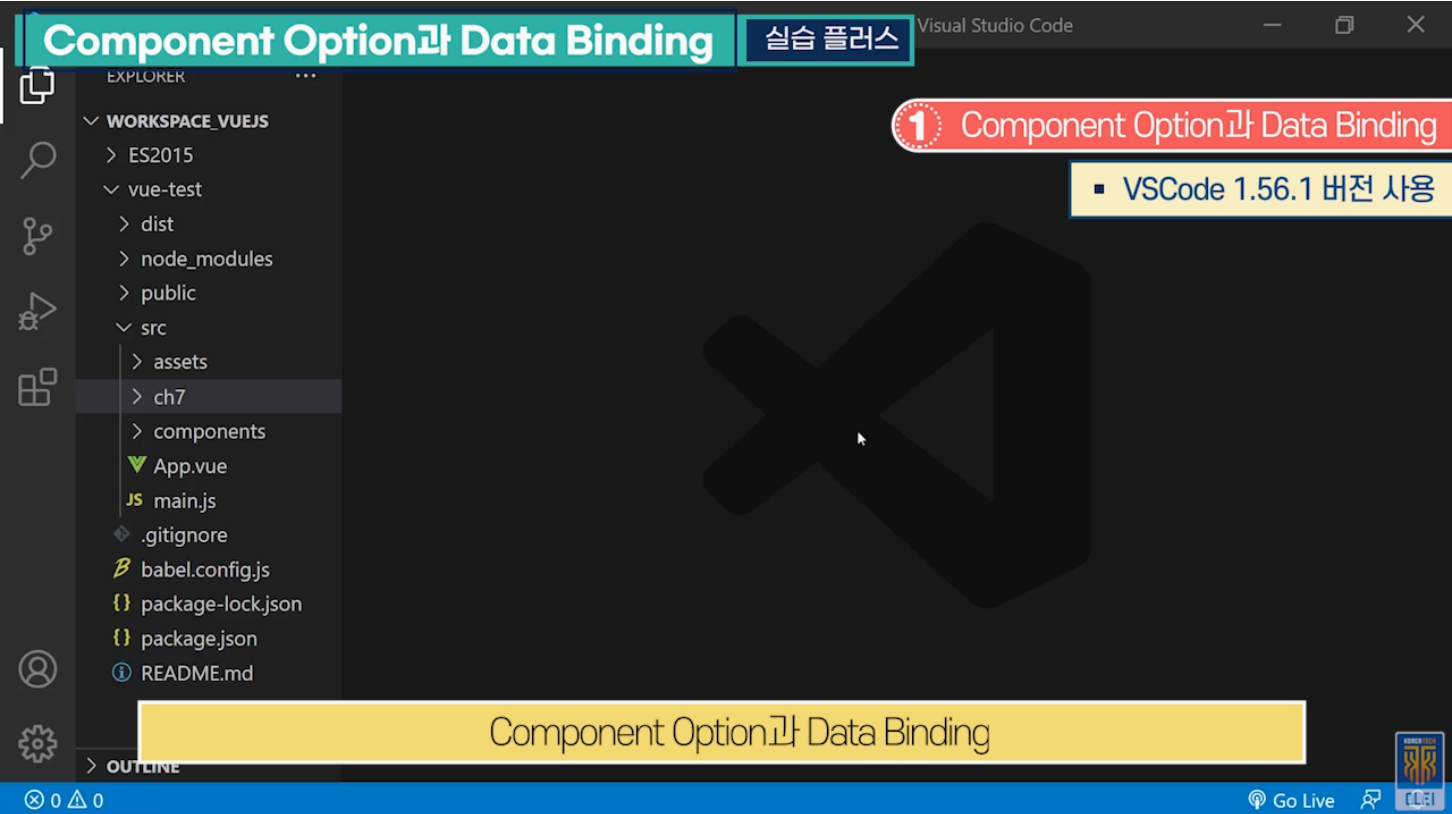
Data Binding

Data Binding

4 표현식 출력

```
{{ number + 1 }}  
{{ ok ? 'YES' : 'NO' }}  
{{ msg.split('').reverse().join('') }}  
  
<div v-bind:id="'list-' + number"></div>  
.....  
data(){  
  return {  
    number: 10,  
    ok: 'YES'  
  }  
}
```

실습 플러스



실습단계
src → ch8 서브 폴더 생성
서브 폴더 생성 후 Vue 파일 생성
Component 정의
script 부분을 먼저 작성하고 template 작성
Vue data 선언
computed : 템플릿 표현식의 복잡함을 줄이기 위해 제공하는 옵션
어떤 함수가 몇 번 호출되었는지 확인하기 위해 console 작성
methods : 함수 스타일로 정의됨
watch 옵션 정의
Vue 화면 구성을 위한 템플릿 정의
data 값 출력
v-once가 있어서 초기 값은 동일하게 출력되지만 변경된 값의 반영 여부가 다름
button 정의
속성 값에 대한 출력이기 때문에 v-bind 이용

실습 플러스

실습단계
선언되어 있는 computed 값을 출력
method 호출하여 값을 출력
테스트를 위해 한 번 더 출력하게 복사해서 지정
computed와 method를 2번 호출하기 위한 의도로, 반복 호출되는지 혹은 한 번만 호출되는지 확인
유저에게 입력을 받고 변경되는 데이터를 watch에서 인식하여 작성된 값으로 변경 가능한지 확인
옵션 정의, 옵션 테스트를 위한 코드 완성
이전에 생성한 ch7의 html, js, vue.config.js 파일 복사하여 이용
main.js 파일 수정하여 사용
테스트 진행
브라우저에 Component 출력
computed : 코드에서 2번 이용해도 데이터가 변경되지 않았기 때문에 한 번만 호출됨 method : 데이터가 변경되지 않아도 2번 호출 가능
초기화된 값 반영

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js `</>` 시작하기

Component Lifecycle과 Props Data



한국기술교육대학교
온라인평생교육원

학습내용

- Component Lifecycle
- Props Data

학습목표

- **Component Lifecycle**에 대해 설명할 수 있다.
- **Props Data**를 이용해 **상위 Component** 데이터 활용을 할 수 있다.

Component Lifecycle

Component Lifecycle

1 Component Lifecycle

- Component는 생성부터 소멸까지 일련의 Lifecycle을 거침
- Create, Mount, Update, Destroy 단계로 나누어짐

1 Create 단계

2 Mount 단계

3 Update 단계

4 Destroy 단계

Component Lifecycle

1 Component Lifecycle

- 각 단계에서의 필요한 작업을 위해 지정된 함수가 호출됨

```
Vue.createApp({  
  data() {  
    return { count: 1 }  
  },  
  created() {  
    console.log('count is: ' + this.count)  
  }  
})
```

Component Lifecycle

Component Lifecycle

2 Create 단계

- Component가 생성되는 단계이며 DOM에 추가되지 않은 단계임
- `beforeCreate()` 함수와 `created()` 함수를 선언할 수 있음

Component Lifecycle

2 Create 단계

`beforeCreate()` data와 events가 준비되지 않은 상태

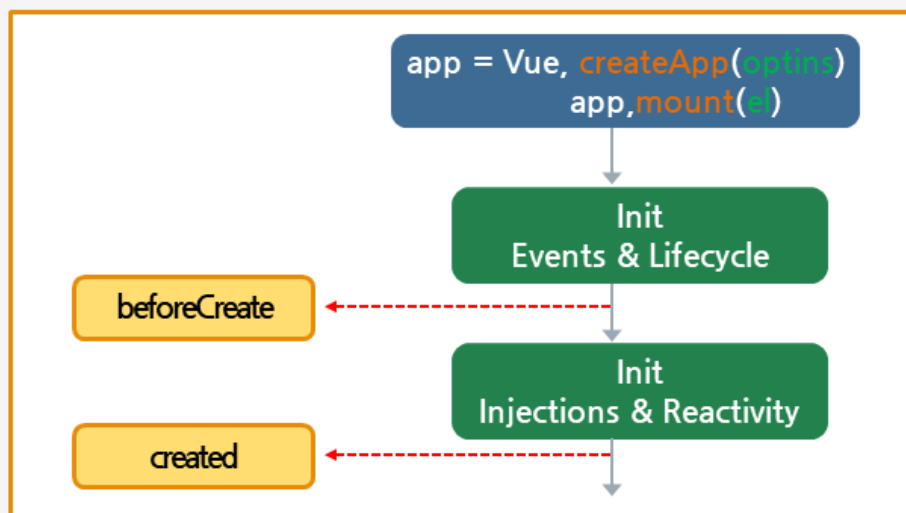
`created()` data, computed, methods, watch 등이 준비된 상태

Component Lifecycle

Component Lifecycle

2 Create 단계

- Created 단계에서는 Mount가 되지 않은 상태이므로 **\$el**을 사용할 수 없음



Component Lifecycle

3 Mount 단계

- Component의 DOM을 준비하여 화면에 출력되는 단계
- `beforeMount()`, `mounted()` 함수가 호출됨

Component Lifecycle

Component Lifecycle

3 Mount 단계

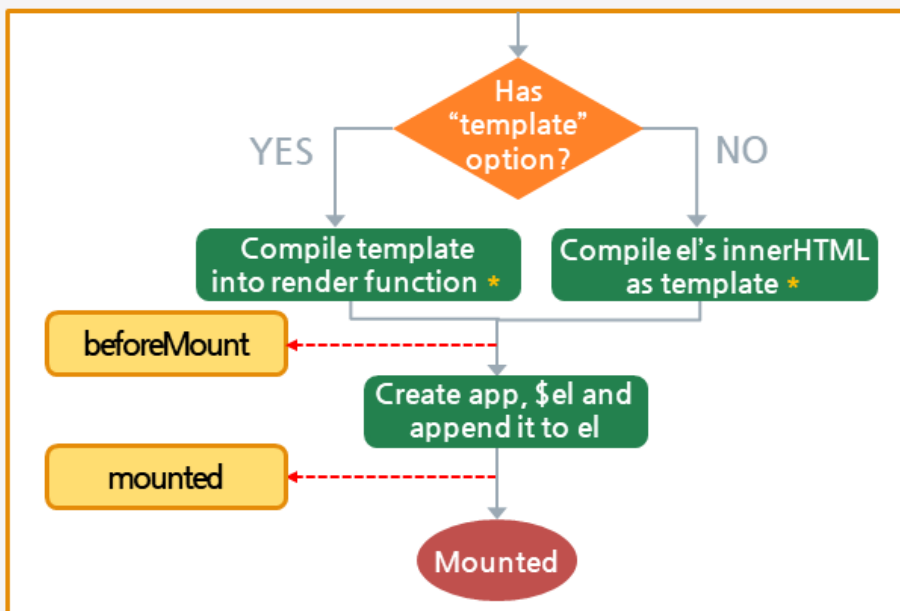
beforeMount() Mount가 시작되기 직전에 호출됨

mounted()

- Mount가 완료된 후 호출됨
- `$el`로 Component의 DOM에 접근할 수 있음

Component Lifecycle

3 Mount 단계



Component Lifecycle

Component Lifecycle

4 Update 단계

- 화면 출력이 변경 될 때 호출됨
- `beforeUpdate()`, `updated()` 함수가 호출됨

Component Lifecycle

4 Update 단계

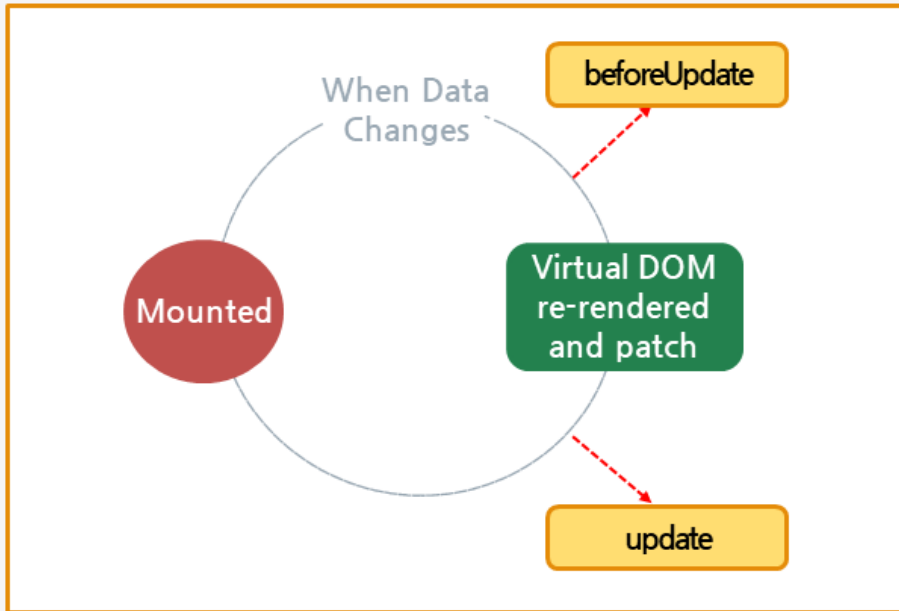
`beforeUpdate()` 데이터가 변경되기 전에 호출됨

`updated()` 데이터 Update가 완료된 후 호출됨

Component Lifecycle

Component Lifecycle

4 Update 단계



Component Lifecycle

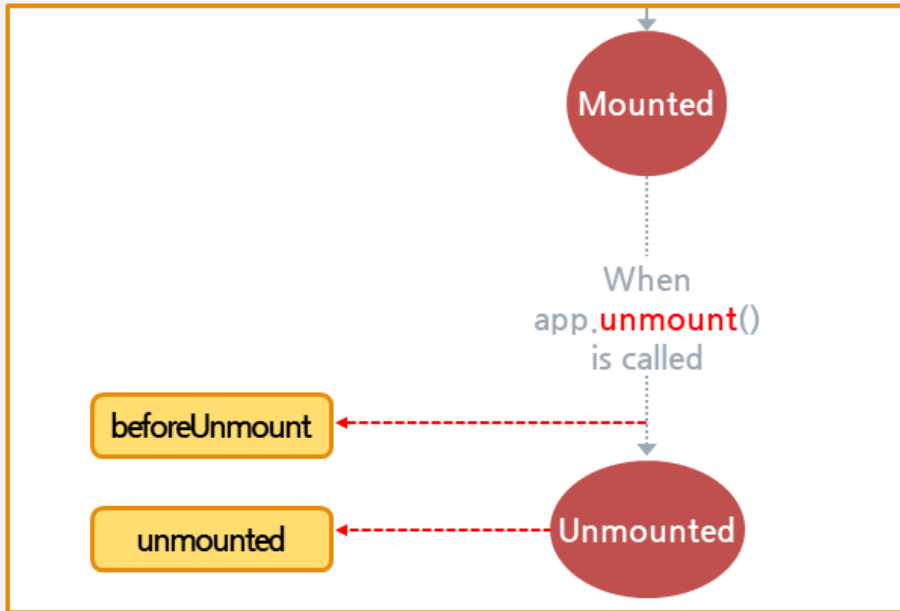
5 Destroy 단계

- Component의 소멸 단계임
- `beforeUnmount()`와 `unmounted()` 함수가 호출됨

Component Lifecycle

Component Lifecycle

5 Destroy 단계

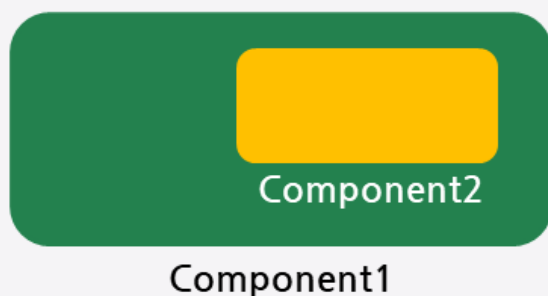


Props Data

Props Data

1 Props란?

- 상위 Component가 전달한 데이터임
- 상위 Component에서 하위 Component를 이용할 때 속성으로 추가한 데이터임



Props Data

2 props option

- 상위에서 속성으로 추가한 데이터를 받기 위한 option
- 상위에서 사용한 속성명을 props에 등록
- props내에 상위의 속성 값이 자동 대입되어 이용됨

Props Data

Props Data

2 props option

○ 상위 Component

```
<Component2 attr1="hello" attr2="10"
attr3="true"></Component2>
```

○ 하위 Component

```
export default {
  props: ['attr1', 'attr2', 'attr3'],
};
```

Props Data

2 props option

○ props option에 대입된 값은 Component에서 this로 이용함

```
<template>
  <div>
    I am sub component!
    <br/>
    super data : {{message}}
    <br/>
    attr1 : {{attr1}}, attr2 : {{attr2}}, attr3 :
    {{attr3}}
  </div>
</template>
```

Props Data

Props Data

2 props option

- props option에 대입된 값은 Component에서 this로 이용함

```
<script>
export default {
  props: ['attr1', 'attr2', 'attr3'],
  data(){
    return {
      message: this.attr1 + ':' + this.attr2 +
        ':' + this.attr3
    }
  }
};
</script>
```

Props Data

3 Type 지정

- props로 전달되는 데이터의 Type 지정
- Object Literal로 props 선언하면서 Value에 Type 지정

Props Data

Props Data

3 Type 지정

- String, Number, Boolean, Array, Object, Function 등 제공

```
props: {  
  attr1: {  
    type: String  
  },  
  attr2: {  
    type: Number  
  },  
  attr3: {  
    type: Boolean  
  }  
},
```

Props Data

4 상위 함수 호출

- 하위 Component에서 상위 Component의 함수 호출
- 상위 Component에서 속성으로 함수를 전달
- 하위 Component에서 **\$props**를 이용해 상위 함수 호출

Props Data

Props Data

4 상위 함수 호출

○ 상위 Component

```
<Sub v-
bind:attr4="superMethod"></Sub>
.....
methods:{
  superMethod: function () {
    console.log("method function
call...");
  },
}
```

Props Data

4 상위 함수 호출

○ 하위 Component

```
props: {
  attr4: {
    type: Function
  },
},
data(){
  this.$props.attr4()
  //.....
}
```

Props Data

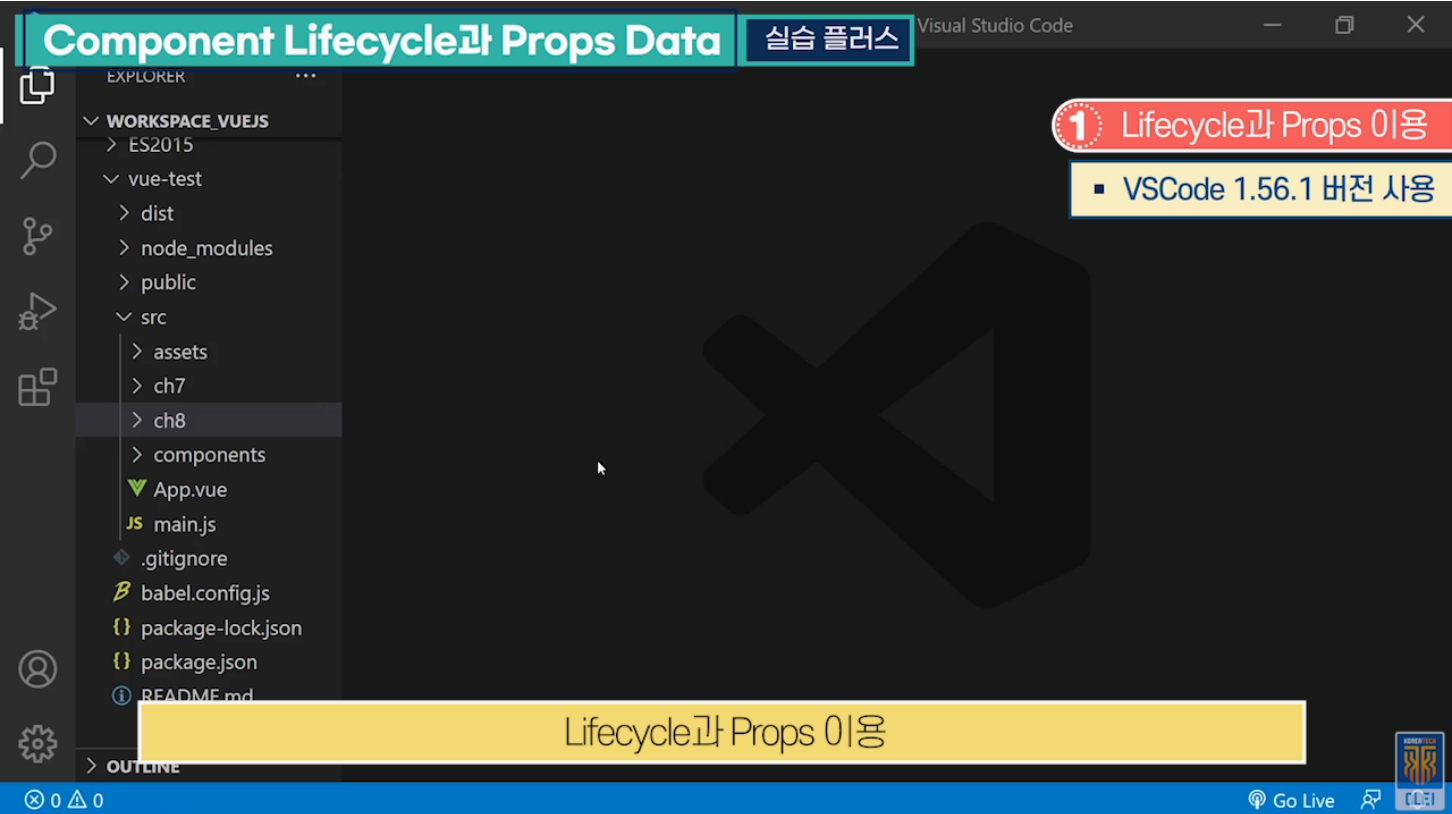
Props Data

5 required

- 필수 전달 데이터도 지정
- Props에 **required: true** 이용

```
props: {  
  attr3: {  
    type: Boolean,  
    required: true  
  },  
}
```

실습 플러스



실습단계
src → ch9 서브 폴더 생성
2개의 Component 정의, 상하 관계로 엮어서 테스트 진행
Sub.vue 생성
다른 Component에 포함돼서 사용될 Component 정의
서브로 등록되어 사용될 Component
전달받을 속성 값 설정
상위 Component의 함수도 전달받을 수 있음
상위 속성 값을 받기 위한 Props 선언 후, 데이터 선언
전달받은 함수와 속성 확인
화면 template 부분 선언, 식별을 위해 'I am sub Component' 작성
super data 출력
상위에서 전달한 데이터를 하위에서 속성 명으로 바로 이용해도 됨
서브 Component 이용하는 Super Component 생성
ch9 → Super.vue 파일 생성

실습 플러스

실습단계
서브 Component를 이용하기 위해 해당 파일 import
자신의 Component 정의
Component 내의 Lifecycle 함수를 추가하여 template 정의한 곳에 있는 HTML 조정
Button Tag를 새로 생성
Button에 들어가는 문자열
사용될 서브 Component 등록
서브에 전달할 함수 선언
template 명시
서브 Component의 속성 값 부여
attr4는 함수 타입, 앞서 정의한 함수 명 superMethod 작성
이전에 생성한 ch8의 html, main.js, vue.config.js 파일 복사하여 이용
Vue의 CLI 명령어 실행
브라우저에 Component 출력
서브에서 Super의 함수 호출은 console로 확인

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Directive



한국기술교육대학교
온라인평생교육원

학습내용

- Directive 선언
- Directive 등록

학습목표

- Directive를 선언하고 Lifecycle 함수를 이용하는 방법에 대해 설명할 수 있다.
- Directive의 Global, Local 등록 사용 방법에 대해 설명할 수 있다.

Directive 선언

Directive 선언

1 Directive

- Component는 동적 UI 구성을 목적으로 하며 태그로 이용됨
- Directive는 속성으로 이용되며 속성이 적용된 DOM에 대한 추가 작업을 목적으로 함

Directive 선언

1 Directive

- Vue.js에서 제공하는 Directive가 있으며 개발자가 직접 만들어 사용할 수 있음

```
<div v-show="false">I am Show</div>
```


Directive 선언

Directive 선언

2 Custom Directive

- Custom Directive는 Object Literal로 만들어짐
- Directive의 Lifecycle 함수를 이용해 Directive에서 실행할 로직을 담음

```
var testDirective = {
  mounted(el) {
    el.focus()
  }
}
```

Directive 선언

3 Directive Lifecycle

- Directive의 Lifecycle은 Directive가 사용되는 Element(Component)의 Lifecycle과 유사하지만 차이가 있음

beforeMount()

Directive가 Element에 바인딩 되면 호출되며 한번만 호출됨

mounted()

Element가 부모 DOM에 삽입되면 호출

beforeUpdate()

Element가 업데이트 되기 전에 호출

Directive 선언

Directive 선언

3 Directive Lifecycle

- Directive의 Lifecycle은 Directive가 사용되는 Element(Component)의 Lifecycle과 유사하지만 차이가 있음

updated()

Element가 업데이트 되면 호출

beforeUnmount()

Element가 마운트 해제되기 전에 호출

unmounted()

Directive가 Element에서 제거되면 호출

Directive 선언

3 Directive Lifecycle

- Directive의 Lifecycle은 Directive가 사용되는 Element(Component)의 Lifecycle과 유사하지만 차이가 있음

```
var testDirective = {
  beforeMount() { },
  mounted() { },
  beforeUpdate() { },
  updated() { },
  beforeUnmount() { },
  unmounted() { },
}
```

Directive 선언

Directive 선언

4 el

- Lifecycle 함수의 매개변수로 전달
- Directive가 사용된 Element 객체

Directive 선언

4 el

- el 객체를 이용해 Directive가 적용된 Element의 조작 작업 가능

```
mounted(el) {  
  el.innerHTML = `  
    el.tagName : ${el.tagName} <br/>  
    el.parentNode.tagName : ${el.parentNode.tagName}  
  <br/>  
`  
}
```

Directive 선언

Directive 선언

5 binding

- Lifecycle 함수의 매개변수로 전달
- Directive와 관련된 다양한 정보 추출 가능함

Directive 선언

5 binding

value

- Directive에 대입된 값
- v-myDirective="2"이면 2값이 대입

oldValue

- 변경되기 전의 값
- beforeUpdate()와 updated()에서만 사용 가능

Directive 선언

Directive 선언

5 binding

arg

- Directive에 적용된 인자
- v-myDirective:foo이면 “foo” 대입

modifiers

- Directive에 적용된 수식어를 포함하는 객체
- v-myDirective.foo.bar이면
{foo:true, bar:true}

Directive 선언

5 binding

```
mounted(el, binding) {
  el.innerHTML = `
    binding.value : ${binding.value} </br/>
    binding.arg : ${binding.arg} <br/>
    binding.modifiers :
    ${JSON.stringify(binding.modifiers)}
  `
}
```

Directive 등록

Directive 등록

1 Global 등록 사용

- application instance에 Directive 등록
- application 전역에서 Directive 이용
- application instance의 directive() 함수로 등록

Directive 등록

1 Global 등록 사용

- 첫 번째 매개변수가 Directive 속성명

```
var app = createApp(MyComponent)
app.directive('testDirective', TestDirective)
app.mount('#app')
```

Directive 등록

Directive 등록

1 Global 등록 사용

- 등록된 Directive 이용

```
<template>
  <div>
    <input v-testDirective />
  </div>
</template>
```

Directive 등록

2 Local 등록 사용

- Directive가 필요한 Component에 등록 사용
- Component의 directives 속성을 이용해 등록

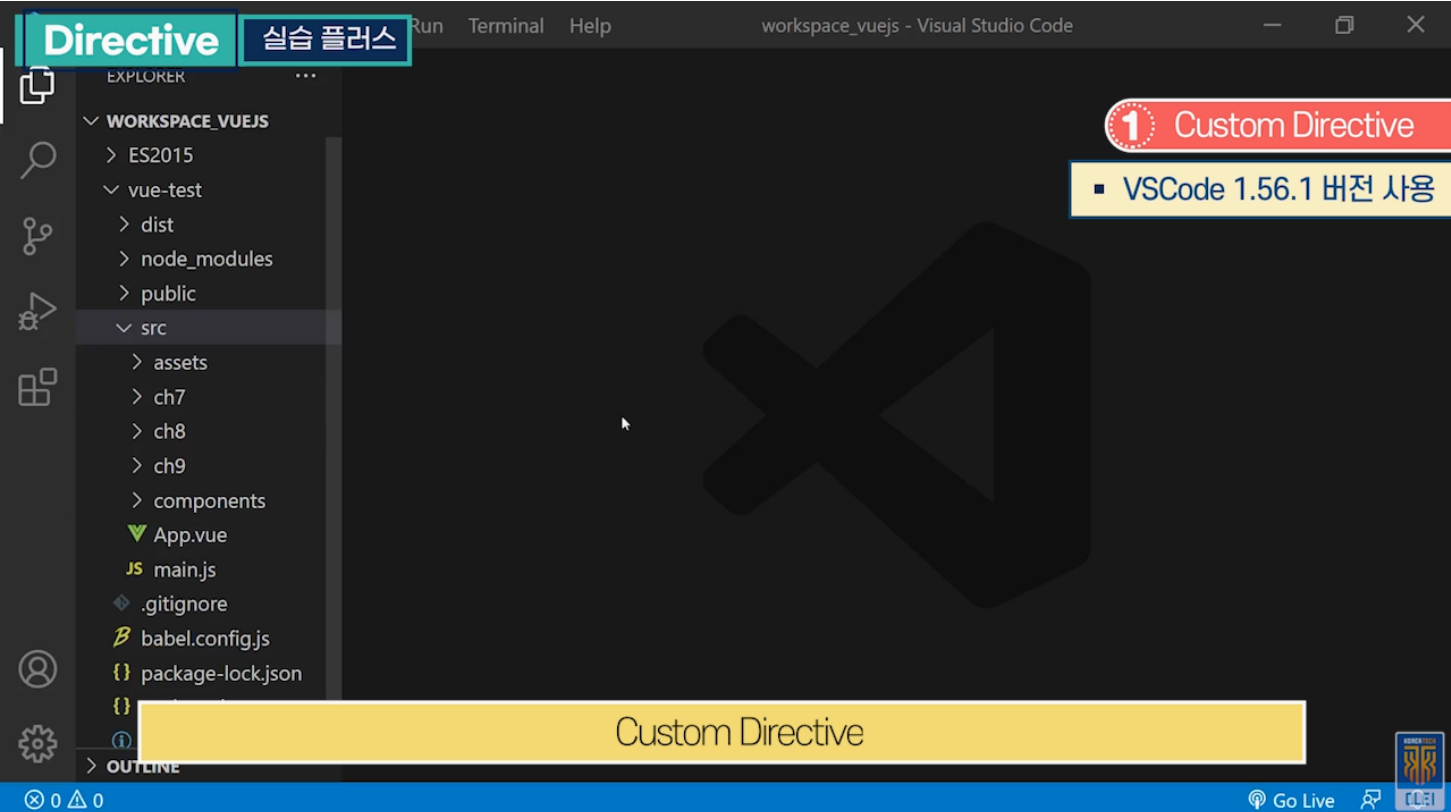
Directive 등록

Directive 등록

2 Local 등록 사용

```
export default {  
  data() {  
    return {  
      msg: "hello",  
      data1: 10  
    };  
  },  
  directives: {  
    myDirective  
  }  
};
```


실습 플러스



실습단계
src → ch10 폴더 생성
생성한 Directive를 Component에서 활용
생성한 폴더 내 myDirective.js 자바스크립트 파일 생성
Directive에 LifeCycle 함수 선언
console로 전달되는 정보 확인
LifeCycle 함수 선언 후 전달받은 Directive와 관련된 정보 확인
Tag에 정보 출력
Directive가 사용된 Tag 명 출력
Directive가 사용된 부모 Tag 정보
Directive에 설정된 값
Directive에 추가된 Argument 정보
Modifiers 값까지 출력
UI 효과 추가
외부에서 이용하기 위해 export 작업

실습 플러스

실습단계
Directive를 이용할 Component를 정의해 줘야 함
사용하고자 하는 Directive 파일 import
Component 정의
Directive 등록 방법 2가지 1. Global로 등록 2. Local로 등록
Local로 등록
HTML Tag에서 사용하기 위해 template 정의
Directive 속성 값 지정
테스트용 Directive에 부가적인 정보 추가
Directive 정의와 Directive를 이용한 Component 정의
이전에 생성한 ch9의 index.html, main.js, vue.config.js 파일 복사하여 이용
Vue 명령으로 실행
브라우저에 Component 출력
전달된 값 확인

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Built-in Directive



한국기술교육대학교
온라인평생교육원

학습내용

- v-bind, v-if, v-for
- v-on, v-model

학습목표

- v-bind, v-if, v-for Directive의 사용 방법에 대해 설명할 수 있다.
- v-on, v-model Directive의 사용 방법에 대해 설명할 수 있다.

v-bind, v-if, v-for

v-bind, v-if, v-for

1 v-bind

- Component의 속성 혹은 Class 값을 바인딩 하기 위한 Directive

v-bind, v-if, v-for

2 class 바인딩

- v-bind을 이용하여 css class 및 style 등록
- v-bind:class="{ className: data }"

v-bind, v-if, v-for

v-bind, v-if, v-for

2 class 바인딩

- data가 true이면 class가 추가되고 false이면 추가되지 않음

```
<div v-bind:class="{ active: isActive }"></div>
.....
data(){
  return {
    isActive: true
  }
}
//출력 : <div class="active"></div>
```

v-bind, v-if, v-for

2 class 바인딩

- 객체 바인딩

```
<div v-bind:class="classObject"></div>
.....
classObject: {
  'active': true,
  'text-danger': true
}
//<div class="active text-danger"></div>
```

v-bind, v-if, v-for

v-bind, v-if, v-for

2 class 바인딩

배열 바인딩

```
<div v-bind:class="[activeClass, errorClass]"></div>
.....
data() {
  return {
    activeClass: 'active',
    errorClass: 'text-danger'
  }
}
//<div class="active text-danger"></div>
```

v-bind, v-if, v-for

2 class 바인딩

3항 연산자 이용 바인딩

```
<div v-bind:class="[isActive ? activeClass : errorClass]"></div>
//<div class="active"></div>
```

v-bind, v-if, v-for

v-bind, v-if, v-for

2 class 바인딩

- 축약형으로 바인딩
- v-bind는 콜론(:)으로 이용 가능함

```
<a v-bind:href="url">google</a>
<a :href="url">google</a>
```

v-bind, v-if, v-for

3 v-if

- 값이 true이면 출력 false이면 출력되지 않음
- v-else, v-else-if와 같이 사용 가능함

```
<div v-if="type === 'A'">A</div>
<div v-else-if="type === 'B'">B</div>
<div v-else-if="type === 'C'">C</div>
<div v-else>Not A/B/C</div>
```


v-bind, v-if, v-for

v-bind, v-if, v-for

4 v-show

- **v-if**와 동일
- display style 값 변경으로 화면에 출력을 제어함
- **v-else**와 같이 사용하지 못함

v-bind, v-if, v-for

4 v-show

- <template> 태그에 사용할 수 없음

```
<div v-show="false">I am Show</div>  
//<div style="display: none;">I am Show</div>
```

v-bind, v-if, v-for

v-bind, v-if, v-for

5 v-for

- 반복 출력
- v-for="item in items"
- v-for Directive가 사용된 Element에는 v-bind:key가 선언되어 있어야 함

```
<li v-for="item in items" :key="item">
  {{ item.message }}
</li>
```

v-bind, v-if, v-for

5 v-for

- index 값 추출

```
<li v-for="(item, index) in items" :key="index">
  {{ index }} - {{ item.message }}
</li>
```

v-bind, v-if, v-for

v-bind, v-if, v-for

5 v-for

- Object Literal를 v-for에 사용하면 객체의 멤버들을 iterator 해 줌

```
<li v-for="value in object" :key="value">
    {{ value }}
</li>
.....
object: {
    firstName: 'John',
    lastName: 'Doe',
    age: 30
}
```

v-bind, v-if, v-for

5 v-for

- key, index 획득 가능
- (value, key, index) In Object

```
<li v-for="(value, key, index) in object" :key="index">
    {{index}} - {{key}} : {{ value }}
</li>
```

v-bind, v-if, v-for

v-bind, v-if, v-for

5 v-for

range

```
<span v-for="n in 2" :key="n">{{ n }} - Hello <br/></span>
```

v-on, v-model

v-on, v-model

1 v-on

- 이벤트(Event) 바인딩
- v-on:click="someFun"
- 축약으로 @click="someFun" 사용가능

```
<button v-on:click="greet">Greet</button>
.....
methods: {
  greet: function (event) {
    .....
  }
}
```

v-on, v-model

1 v-on

- parameter 전달

```
<button v-on:click="warn('hello', $event)">
  Submit
</button>
.....
methods: {
  warn: function (message, event) {
    if (event) event.preventDefault()
    .....
  }
}
```

v-on, v-model

v-on, v-model

2 이벤트(Event) Modifier

- 다양한 modifier 등록으로 쉽게 이벤트 제어 가능

```
<form v-on:submit.prevent="onSubmit"></form>
```

v-on, v-model

2 이벤트(Event) Modifier

.stop

event.stopPropagation()

.prevent

event.preventDefault()

.capture

- 이벤트를 capture 하겠다는 의미
- 하위 element에서 이벤트가 발생하여도 capture가 선언된 곳에서 이벤트를 먼저 처리하기 위해서 사용

v-on, v-model

v-on, v-model

2 이벤트(Event) Modifier

.self

자신에게 발생한 이벤트만
처리하겠다는 의도

.once

한번만 이벤트 허용

v-on, v-model

2 이벤트(Event) Modifier

- 이벤트 수식을 등록하면서 chain 형태로 여러 개 등록 가능

```
<a v-on:click.stop.prevent="doThat"></a>
```

v-on, v-model

v-on, v-model

3 키(Key) Modifier

- 키 이벤트 처리시 이벤트가 발생한 key 지정

```
<input @keyup.enter="keyEvent">
```

v-on, v-model

4 v-model

- Two-way Data Binding

```
<input v-model="name" placeholder="input name">
```


v-on, v-model

v-on, v-model

4 v-model

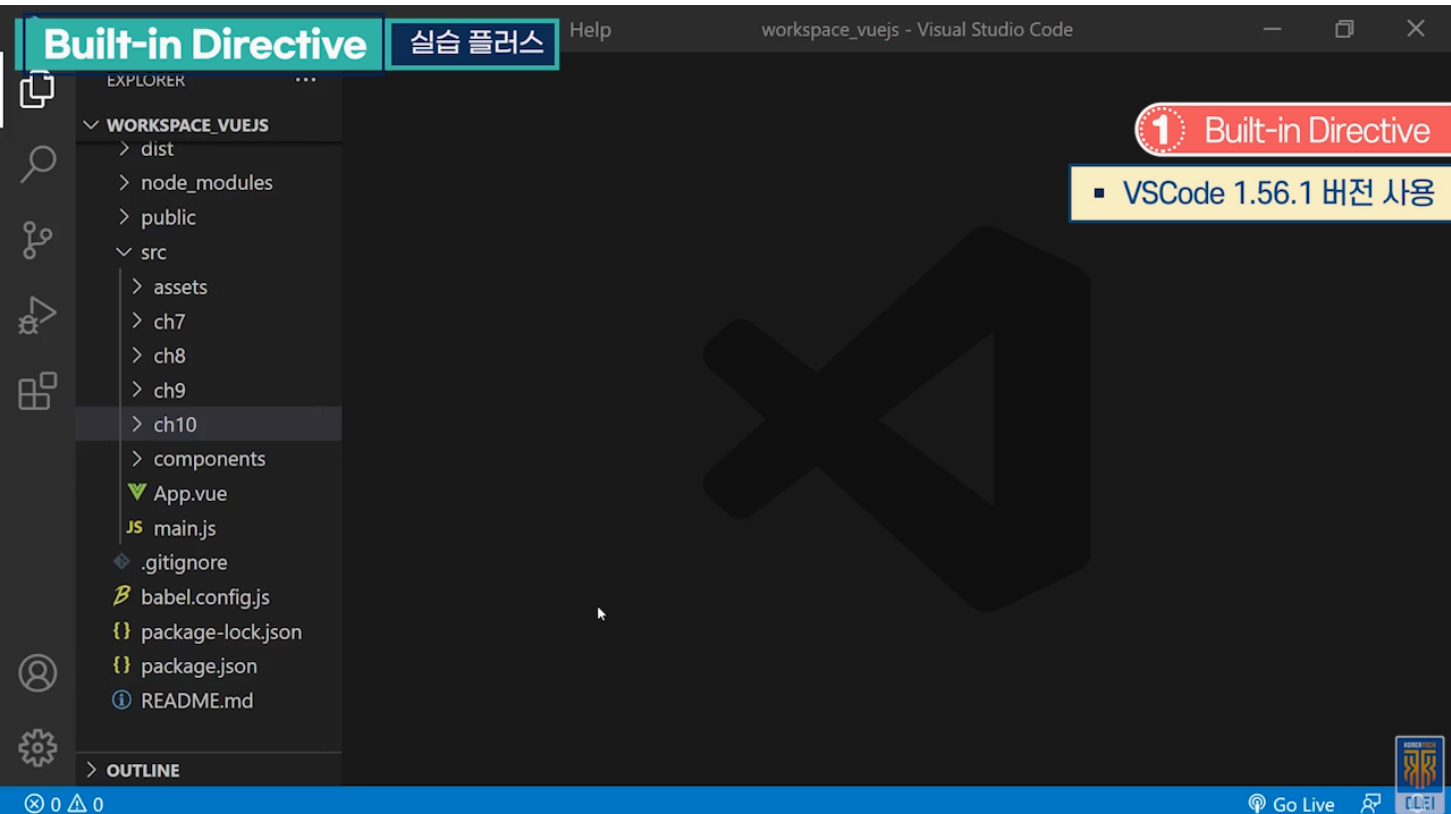
- checkbox의 경우 v-model에 의해 바인딩 되는 데이터는 true/false임
- 여러 개의 checkbox가 있고 check 된 value 값을 문자열 배열로 저장하는 것도 가능

v-on, v-model

4 v-model

```
<input type="checkbox" value="kkang" v-model="checkedNames">  
<input type="checkbox" value="kim" v-model="checkedNames">  
<input type="checkbox" value="lee " v-model="checkedNames">
```

실습 플러스



실습단계

src → ch11 서브 폴더 생성

Component 내에서 Built-in Directive 테스트

생성한 폴더 내 Test.vue 파일 생성

script 영역 정의

정의한 데이터를 Built-in Directive로 HTML에 Binding

v-if Directive 테스트에 사용됨

v-for 테스트를 위한 데이터 선언

methods 지정

로그인 타입을 변경할 때 호출되는 함수

Event 시 호출되는 함수 정의

Event Callback 함수 추가 정의

정의한 데이터와 함수를 이용하기 위해 템플릿에서 내장 Directive 사용

v-bind 테스트

데이터 값으로 Binding 하는 style 정의

실습 플러스

실습단계
v-bind:를 :으로 축약 가능
v-if 테스트
현재 loginType 데이터가 username일 경우
template을 복사하여 추가로 v-else 정의
Event 추가
v-for 테스트
선언되어 있는 변수 개수 만큼 li Tag 반복됨
한 번 반복되면서 자동으로 들어간 index 값과 데이터 출력
배열 데이터 이외에 object로 v-for 선언 가능
다양한 event 테스트
input Tag 사용하여 KeyEvent 테스트
Two Way Data Binding
이전에 생성한 ch10의 html, main.js, vue.config.js 파일 복사하여 이용
Vue 명령으로 실행
브라우저에 Component 출력
console 창에서 각 event의 차이 확인 가능
첫 번째 버튼, console에 출력되지만 Refresh 상태
두 번째 버튼, Refresh 되지 않음

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Router



한국기술교육대학교
온라인평생교육원

학습내용

- Single Page Application
- vue-router

학습목표

- Single Page Application에 대해 설명할 수 있다.
- vue-router 사용 방법에 대해 설명할 수 있다.

Single Page Application

Single Page Application

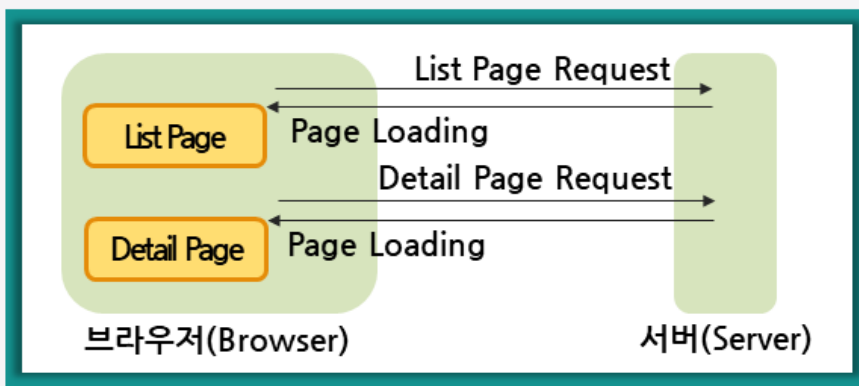
1 Single Page Application

- Single Page Application(SPA)는 하나의 페이지에서 애플리케이션의 여러 화면을 제공하는 웹 애플리케이션을 의미함
- Multi Page Application의 반대 개념으로 이용됨

Single Page Application

1 Single Page Application

Multi Page Application의 구조

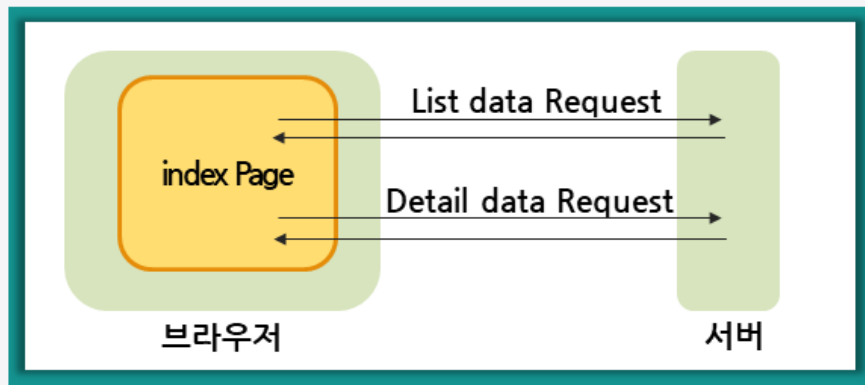


Single Page Application

Single Page Application

1 Single Page Application

Single Page Application의 구조



Single Page Application

2 Routing



Routing

- 특정 화면에서 다른 화면으로의 이동을 의미함
- Single Page Application이 되려면 Routing이 물리적인 파일 로딩에 의한 화면전환이 아닌 하나의 페이지내에서 논리적인 화면전환이 되어야 함

Single Page Application

Single Page Application

2 Routing



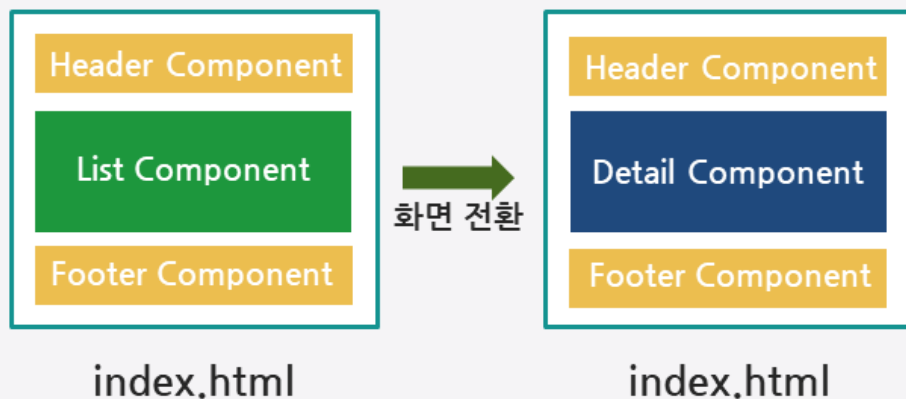
Routing

- 각 화면(혹은 화면의 구성요소)이 Component로 개발되어 있어야 함

Single Page Application

2 Routing

- Client Side(Javascript)에서 어떤 조건에서 어떤 화면(Component)이 출력되어야 하는지 결정되어야 함
- 이 부분을 지원하기 위한 라이브러리가 Router임



vue-router

vue-router

1 vue-router 기본

① Router 설치

- Router를 이용하기 위해서는 vue-router를 설치해야 함
- vue-router는 Vue.js의 공식 라이브러리임

```
>npm install vue-router@4
```

vue-router 4 버전

Vue.js 3 버전을 위한 Router 라이브러리

vue-router 3 버전

Vue.js 2 버전을 위한 Router 라이브러리

vue-router

1 vue-router 기본

② Routing 조건

- URL의 Path 조건
- http://localhost:8080/이라면 Routing조건은 /임
- http://localhost:8080/login이라면 Routing 조건은 /login임

vue-router

vue-router

1 vue-router 기본

2 Routing 조건

→ Routing 조건을 명시한 배열을 준비함

path	Routing 조건
component	Routing 조건이 맞을 때 출력할 Component

vue-router

1 vue-router 기본

2 Routing 조건

```
const routes = [
  { path: '/', component: Home },
  { path: '/login', component: LoginPage },
  { path: '/detail', component: DetailPage },
]
```

vue-router

vue-router

1 vue-router 기본

③ Router Instance 생성

- ... createRouter() 함수를 이용해 Router Instance를 생성함
- ... routes option에 Routing 조건을 등록해 줌

```
const router = createRouter({  
  history: createWebHistory(),  
  routes  
})
```

vue-router

1 vue-router 기본

④ Router Instance 사용 선언

- ... application instance의 use() 함수로 router instance 등록

```
app.use(router);
```

vue-router

vue-router

1 vue-router 기본

⑤ router-view component 등록

... Router에 의해 출력되는 Component는 <router-view/>에 출력됨

```
<router-view></router-view>
```

vue-router

1 vue-router 기본

⑥ router-link

... 화면에 출력되는 라우팅을 위한 링크

```
<router-link to="/login">Go to Login</router-link>
```

vue-router

vue-router

1 vue-router 기본

⑦ \$router.push

- Routing에 의해 출력되는 Component에서는 \$router 객체로 Routing과 관련된 다양한 데이터 및 함수 이용
- \$router.push() 함수로 특정 path 조건의 Component로 이동

```
this.$router.push('/login')
```

vue-router

2 vue-router parameter

① parameter 선언

- parameter 값을 routing 조건에 추가할 때는 path에 **콜론(:)**을 추가해 명시함

```
{ path: '/detail/:title', component: DetailPage }
```

vue-router

vue-router

2 vue-router parameter

1 parameter 선언

- path 조건의 콜론(:)은 **임의의 문자열 하나**를 의미함
- 콜론(:) 뒤 문자열은 임의 문자열이며 전달되는 데이터의 key 값으로 사용됨
- 콜론(:)에 의한 조건은 하나의 path 내에 여러 번 나올 수 있음

vue-router

2 vue-router parameter

1 parameter 선언

path	Matched path	전달 데이터
/users/:username	vue-router parameter	vue-router parameter
/users/:username/posts/:postId	/users/kim/posts/123	{username: 'kim', postId: 123}

vue-router

vue-router

2 vue-router parameter

② parameter 획득

- ... Routing 조건에 의해 실행되는 Component에서는 parameter 값을 `$route.params` 객체로 획득함

```
title : {{$route.params.title}}
```

vue-router

3 vue-router 중첩

① Nested Routing

- ... Routing 조건이 복잡한 경우 Routing 조건을 중첩에 의해 선언이 가능함
- ... Routing 조건에 의해 실행된 Component부터 다시 Routing 조건을 판단함

vue-router

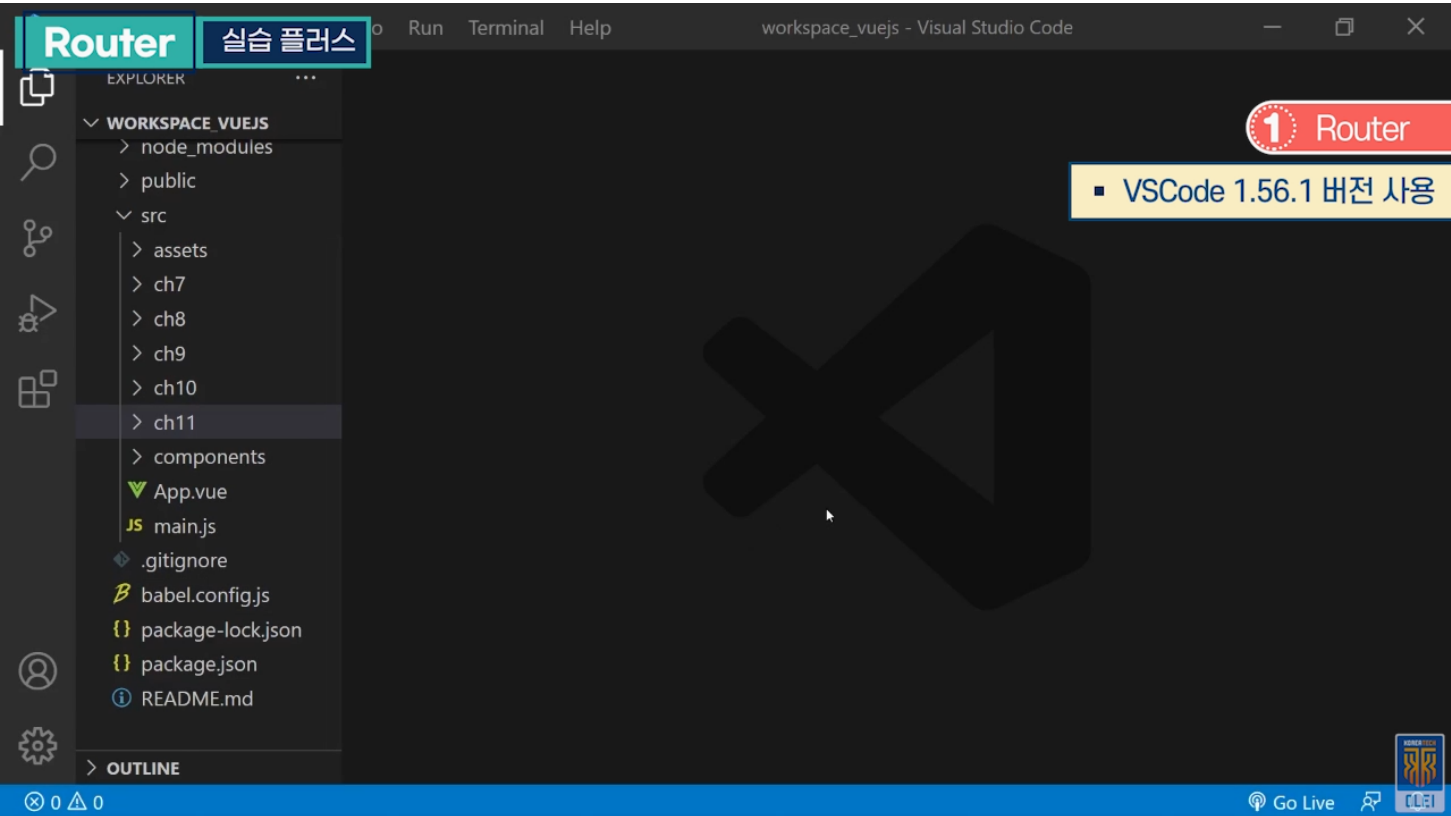
vue-router

3 vue-router 중첩

① Nested Routing

```
const router = new VueRouter({
  routes: [
    { path: '/user/:id', component: User,
      children: [
        { //user/:id/profile is matched
          path: 'profile',
          component: UserProfile
        },
        { //user/:id/posts is matched
          path: 'posts',
          component: UserPosts
        }
      ]
    }
  ]
})
```


실습 플러스



실습단계
src → ch12, Router를 테스트하고자 하는 것이 주목적임
Router 설치, npm CLI 명령어를 통하여 Router 모듈 추가 설치
프로젝트에서 npm 명령을 내려주면 됨
ch12 폴더에 Component 정의, Router : Component를 교체하면서 화면을 전환하고자 하는 것이 주목적
New → File에서 첫 화면으로 쓰고자 하는 Component, Home.vue 선언
template, script 정의 및 작성
배열 데이터로 목록 화면 출력, 목록에 각각의 문자열을 출력시키기 위해서 작성한 것
이벤트 발생에 의해서 호출되며, Router로 화면 전환
화면을 꾸미기 위해 template 정의
배열 데이터를 v-for로 나열, 항목이 나열되고 클릭할 때 event 함수가 호출되게 화면 구성
ch12 폴더에 Login.vue 정의
Component의 이름 정도만 화면에 출력되게끔 작성
ch12 폴더에 Detail.vue 작성

실습 플러스

실습단계
화면에 식별을 위해 title 작성
Home.vue의 Component가 나오고 항목을 눌렀을 때 Detail 화면이 나오도록 함 → 이전 화면에서 전달된 데이터를 받을 수 있도록 테스트함
Router의 내장객체 : 내장객체를 이용하여 전달한 데이터를 가지고 있는 변수 params로 데이터 획득
세 개의 Component를 Router에 등록, main.js에서 작업
이전에 생성한 ch11의 index.html, main.js, vue.config.js 파일 복사하여 이용
Router에서 제공되는 API를 import 받음, Component도 import하여 Home으로 정의
Component 간 화면관리, 화면전환을 Router로 함
Router에 등록할 정보 준비
첫 번째 path 정보, 어떤 path가 들어왔을 때 Component를 실행시킬 것인지에 대한 정보
‘/’로 들어왔을 때 실행될 Component
/login으로 path가 들어오면 Login Component 출력
Detail path의 ‘:’ 뒤에 임의의 단어 하나는 title이란 변수로 저장, Component는 DetailPage
정보가 Router에 추가되어야 하므로 API로 Router 생성
Router에 정보가 포함되므로 Router에 의해 정보에 맞는 화면이 출력됨
Router 객체가 애플리케이션에 등록되어 있는 상태
index.html에 Router가 나와야 할 위치 지정
테스트를 위해 링크 준비
화면에 go home이 나옴, 클릭했을 때 path 정보 값이 ‘/’로 바뀌고 Router가 캐치하여 ‘/’에 의해 Component 출력
Vue 명령으로 실행
브라우저에 Component 출력
Router에 의한 Component가 출력된 부분, Router에 의해서 Detail Page Component로 전환됨

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Vuex



한국기술교육대학교
온라인평생교육원

학습내용

- State Management
- Vuex Overview

학습목표

- 상태 관리에 대한 개념을 설명할 수 있다.
- Vuex의 기초 사용 방법을 설명할 수 있다.

State Management

State Management

1 상태(State)란



상태(State)의 정의

- 애플리케이션의 데이터
- Component 내에서만 유지되고 관리되어야 하는 데이터를 상태라 표현하지는 않음
- 애플리케이션 전역에서 이용되는 데이터

State Management

2 상태 관리(State Management)란



상태 관리(State Management)의 특징

- 여러 Component로 구성된 애플리케이션에서는 하나의 상태가 하나의 Component에서만 이용되지 않음
- 애플리케이션 관점에서 상태를 **체계적으로 유지 관리**해 주어야 함
- 애플리케이션을 상태 중심으로 설계 개발함으로써 **애플리케이션의 데이터와 데이터의 흐름을 명료**하게 할 수 있음

State Management

State Management

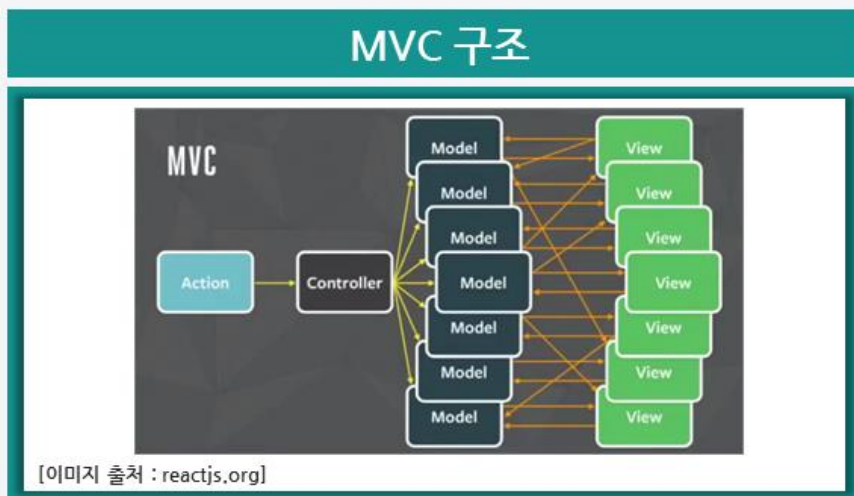
3 MVC 구조에서의 데이터 흐름

- MVC 모델은 기능의 분리가 초점임
- 애플리케이션의 상태를 중심으로 한 흐름이 아님

State Management

3 MVC 구조에서의 데이터 흐름

- 애플리케이션의 데이터가 **양방향으로 이용**되어 **복잡도가 증가**됨

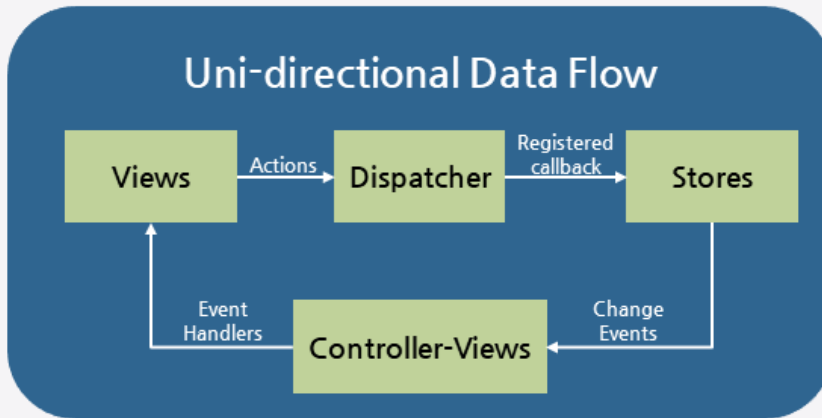


State Management

State Management

4 상태 관리 패턴의 데이터 흐름

- 애플리케이션의 데이터는 **단방향으로만** 이용되어야 함



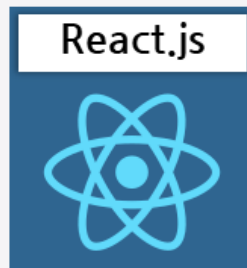
[이미지 출처 : reactjs.org]

State Management

5 상태 관리 프레임워크



NGXS



Redux, Context



Vuex

Vuex Overview

Vuex Overview

1 Vuex

- Vue.js를 위한 상태 관리 프레임워크

Vuex4

Vue.js 3 버전을 위한 프레임워크

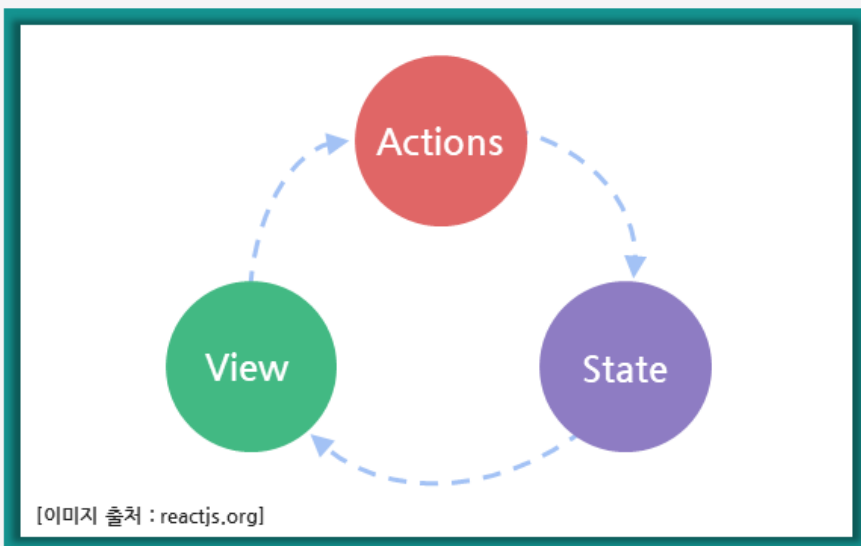
Vuex3

Vue.js 2 버전을 위한 프레임워크

```
>npm install vuex@next
```

Vuex Overview

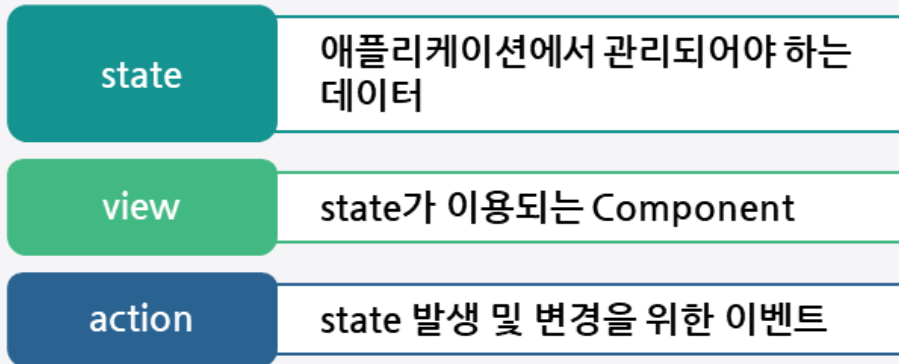
2 One way data flow



Vuex Overview

Vuex Overview

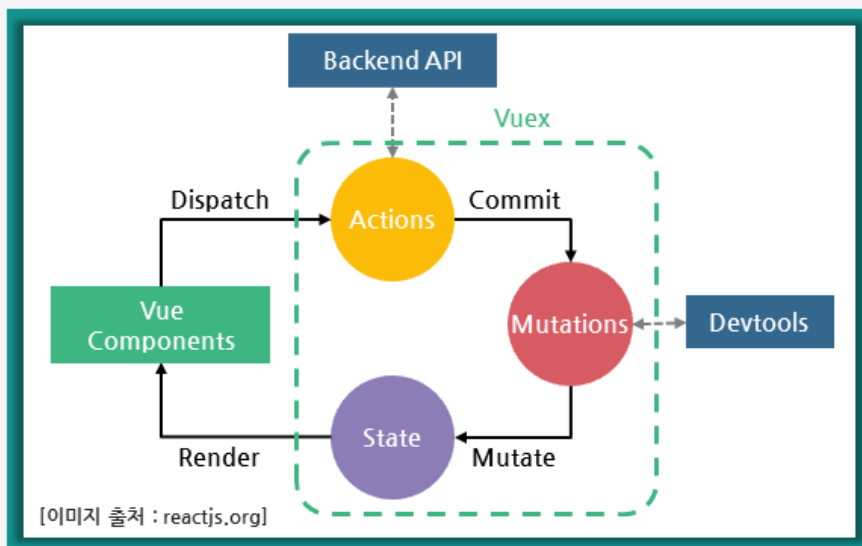
2 One way data flow



Vuex Overview

3 Vuex의 Flow

mutation : state 변경



Vuex Overview

Vuex Overview

4 Store Instance

- Store Instance는 **state, actions, mutations**로 구성됨
- actions과 mutations는 함수로 작성됨
- state는 Object Literal임

Vuex Overview

4 Store Instance

```
const store = createStore({
  state() {
    return {
      items: []
    }
  },
  actions: {
    addItemAction({ commit }, item) {
      item.id=count++
      commit('addItemMutation', item)
    }
  },
  mutations: {
    addItemMutation(state, item ) {
      state.items.push(item)
    }
  }
})
```

Vuex Overview

Vuex Overview

5 Store Instance 등록

- application instance의 use() 함수로 application에 등록하여 사용함

```
app.use(store)
```

Vuex Overview

6 action 발생

- 애플리케이션 내에서 Vuex의 구성요소는 this.\$store 객체로 접근이 가능함
- action은 함수이며 함수 이름을 this.\$store.dispatch() 함수에 대입하여 호출함

Vuex Overview

Vuex Overview

6 action 발생

- dispatch 함수를 호출하면서 데이터를 매개변수로 전달할 수 있음

```
this.$store.dispatch('addItemAction', data)
```

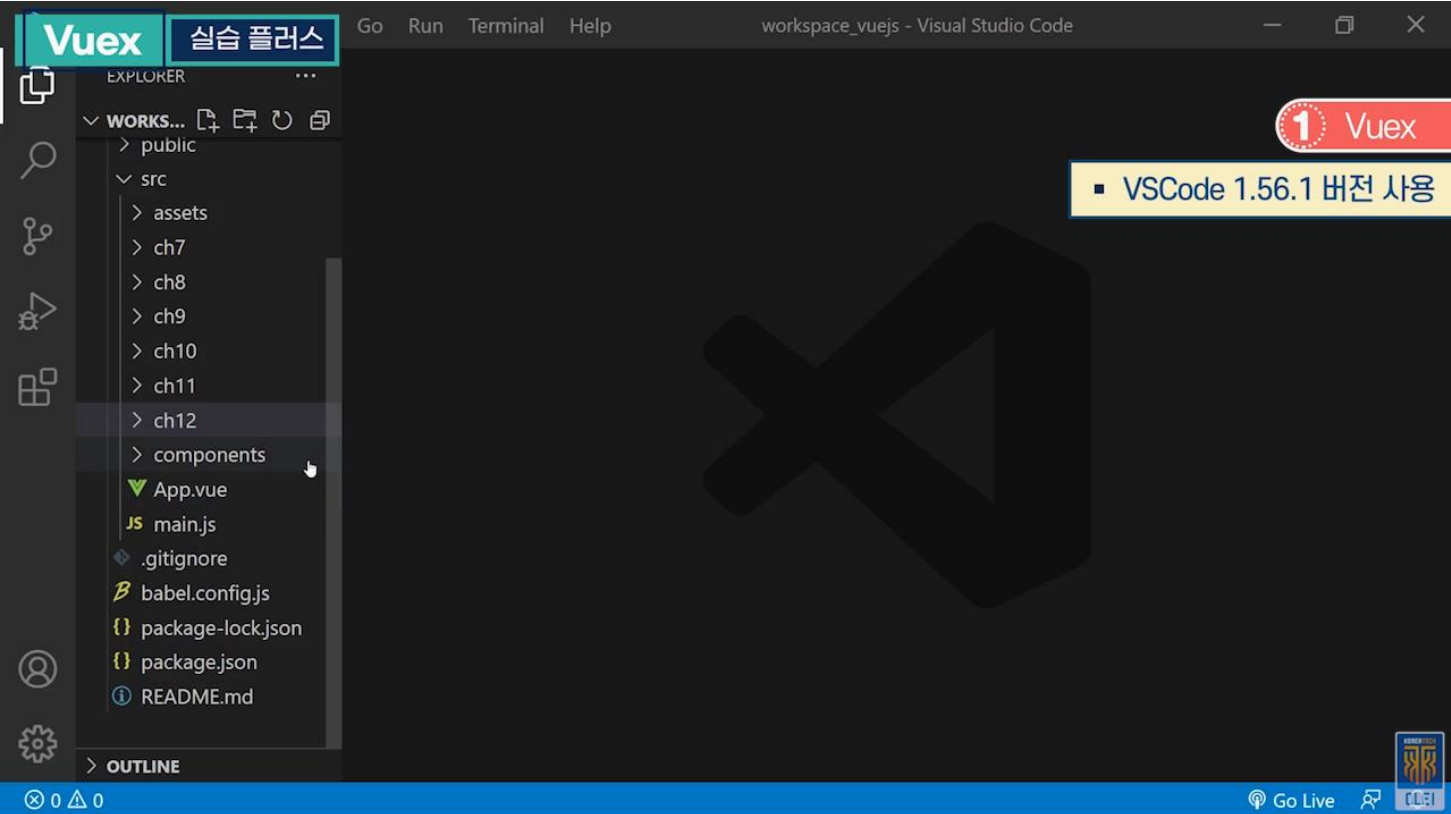
Vuex Overview

7 state 이용

- action 발생에 의해 생성/변경된 state 데이터는 `this.$store.state` 객체로 획득이 가능함

```
this.$store.state.items
```

실습 플러스



실습단계
src → ch13
이전에 생성한 ch11 폴더의 index.html, main.js, vue.config.js 파일 복사하여 이용
Vuex 테스트가 목적이므로 Vuex 먼저 설치함
vue-test 위치에 설치, npm install로 module 설치
Vuex를 활용할 수 있는 Test.vue 파일 생성
main.js 부분 정의, Vue가 import되어 있음, Vuex도 import
Vuex에 state나 action을 추가하기 위한 정보 포함
state는 count 값 하나 유지, state 변경을 위한 action 선언
increase 함수를 호출해서 state 값 변경
부가적인 업무를 수행할 수 있음, 최종 데이터 변경은 매개변수로 전달되는 commit 이용
Component에서 데이터 변경을 하기 위해 함수를 호출하면 action에서 추가 업무를 진행 후, commit 명령 선언
commit 명령에 의해 increase 함수를 호출하여 매개변수 값을 변경하는 구조
애플리케이션에 Vuex 정보 등록

실습 플러스

실습단계
Vuex를 Component에서 이용, 만들어 놓은 Vue Component 파일에서 정의
Component 함수가 호출됐을 때 Vuex 쪽에 action을 발생 → Vuex에 의해서 이용되는 Component 내장 객체
Vuex에 유지하는 데이터를 가져오기 위해 computed 사용
Vuex 데이터를 얻을 수 있음
현재 Vuex에 유지되고 있는 데이터 출력, 데이터를 변경할 수 있는 버튼 추가
유저가 화면에서 버튼을 눌렀을 때 increase 함수가 호출되고 함수 내부에서 Vuex에 action을 발생시켜서 값 변경
Vue 명령으로 실행
count 값이 출력되어 있고 increase 버튼이 있음

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

Vuex 활용



한국기술교육대학교
온라인평생교육원

학습내용

- Module, State
- Action, Getter

학습목표

- **Module**과 **State** 사용 방법에 대해 설명할 수 있다.
- **Action**과 **Getter** 사용 방법에 대해 설명할 수 있다.

Module, State

Module, State

1 Module



Module

- Store에는 **actions, mutations, state**가 등록됨
- 애플리케이션 내에 여러 개 Store 이용 가능

Module, State

1 Module



Module

- 애플리케이션 규모가 크면 상태별로 Store를 여러 개 이용하는 것이 효율적임
- Store를 모듈화해서 여러 개 등록하고 구분해서 사용하는 기법

Module, State

Module, State

2 모듈선언

- 모듈은 **state, actions, mutations**를 가지는 **Object Literal**임

Module, State

2 모듈선언

namespaced: true

- 모듈을 이름으로 구분해서 사용하겠다는 의미

```
const countStore = {  
  namespaced: true,  
  state() { },  
  actions: { },  
  mutations: { }  
}
```

Module, State

Module, State

3 Module 등록

- Object Literal로 선언된 모듈을 modules 속성으로 등록해서 사용함

```
const store = createStore({
  modules: {
    countStore: countStore,
    todoStore: todoStore
  }
})
```

Module, State

4 Module의 이용

Module의 Action 이용

- Action 함수 앞에 모듈명을 추가해 이용함

```
this.$store.dispatch('countStore/increase')
```

Module, State

Module, State

4 Module의 이용

Module의 State 이용

- `this.$store.state` 뒤에 모듈명을 추가해 이용함

```
this.$store.state.countStore.count
```

Module, State

5 State

- 애플리케이션의 상태를 의미함
- Object Literal로 선언됨

```
state() {  
  return {  
    count: 0  
  }  
},
```

Module, State

Module, State

6 this.\$store로 이용

```
export default {
  computed: {
    count() { return this.$store.state.test.count; }
  },
};
```

Module, State

7 mapState 활용

- state 값을 쉽게 이용하기 위한 일종의 Helper
- mapState에 함수를 등록하면 매개변수로 state를 전달해 줌

```
import { mapState } from 'vuex'
export default {
  computed: mapState({
    myCount: state => state.test.count,
    myCount2 (state) {
      return state.test.count
    }
  })
};
```

Module, State

Module, State

7 mapState 활용

- state 값을 직접 mapState를 이용해 획득 가능

```
...mapState('test', ['count']),
```

Action, Getter

Action, Getter

1 Action

- 애플리케이션의 State 값 변경을 위해 호출되는 함수

```
actions : {  
  increase({commit }) {  
    commit('increase')  
  },  
}
```

Action, Getter

2 Action 함수 호출

- this.\$store의 dispatch 함수를 이용해 호출함

```
this.$store.dispatch('test/testAction', 100)
```

Action, Getter

Action, Getter

2 Action 함수 호출

- mapActions 함수 이용
- Action 함수를 쉽게 이용하기 위한 Helper

```
... mapActions("test", ["testAction"])
```

Action, Getter

3 Getter

- 필수 구성요소는 아님
- Getter는 state를 이용하기 위한 함수

Action, Getter

Action, Getter

4 Getter 선언

- 매개변수로 state가 전달되며 두번째 매개변수로 다른 Getter를 이용할 수 있는 getters가 전달됨

```
const getters = {
  doneTodos: state => {
    return state.todos.filter(todo => todo.done)
  },
  doneTodosCount: (state, getters) => {
    return getters.doneTodos.length
  }
}
```

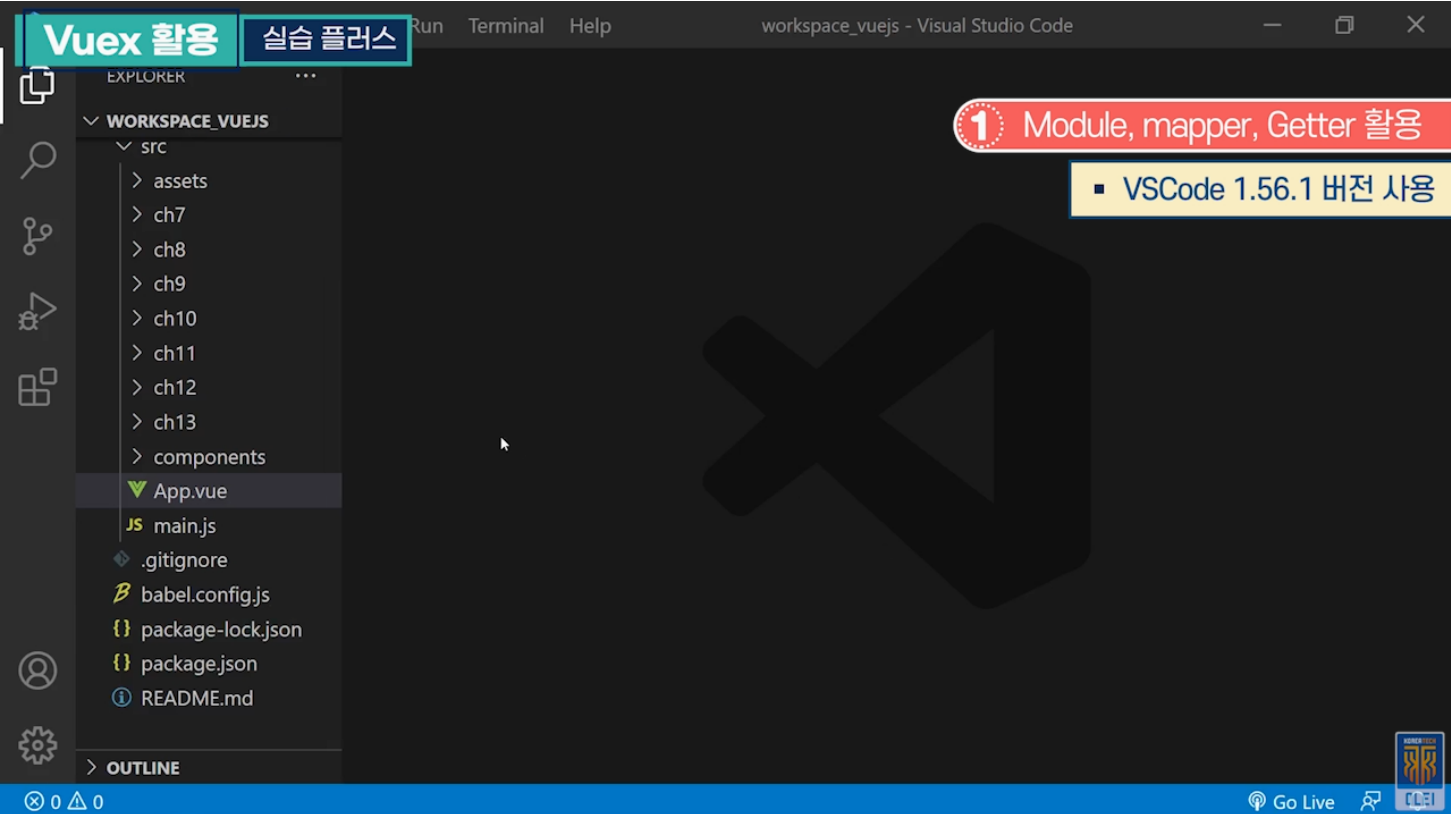
Action, Getter

5 Getter 이용

- mapGetters() 함수로 Getter를 이용함

```
import { mapState, mapGetters } from "vuex";
export default {
  computed: {
    ...mapGetters('test', {
      doneTodosCount: 'doneTodosCount'
    })
  },
}
```

실습 플러스



실습단계
src → ch14 폴더 생성, 서브 폴더 store 생성
Vuex 이용할 때 나오는 많은 구성요소들을 담기 위한 서브 폴더
store 폴더에 countStore.js 파일 생성
애플리케이션 전역에서 유지되어야 되는 데이터 중 count값과 관련되어 있는 데이터를 관리하기 위한 자바스크립트
애플리케이션 전역 데이터를 구분하기 위해 namespaced 선언
countStore에 의해서 관리되는 데이터를 이름 값으로 구분하여 이용하기 위한 설정
관리 되어야하는 데이터를 state에 등록
action 발생 시 호출될 함수를 actions에 등록, action 함수가 호출됐을 때 자체적인 업무 로직을 함수에 넣음
최종 데이터 변경을 위해 commit 함수 호출, mutation의 increase 함수 호출 → mutation 등록
mutation에 increase 함수가 호출됐을 때, 자체적으로 유지하고 있는 state 데이터 변경
Component에서 이용 편의성을 위해 getter라는 구성요소가 추가될 수 있음
store의 state 데이터를 component에서 이용, getter 없이도 이용 가능하지만 getter를 등록할 경우 편의성 있음
store → todoStore.js 파일 생성

실습 플러스

실습단계
state 데이터 변경을 위해 호출되는 함수가 action 함수
새로운 todo 글이 함수가 호출하면서 전달됨, 식별을 위한 id 값을 유지하기 위해서 기존 id 값에서 1을 더함
state 변경을 위한 commit 함수 호출
commit 함수에서 전달한 데이터가 두 번째 매개변수로 전달
외부에서 이용 가능하도록 export 시킴
Vuex 이용을 위한 Component 정의
Ch14 → AddComponent 파일 생성
API를 먼저 import 받음
두 개의 데이터는 애플리케이션 전역이 아니라 Component 내부에서만 유지하는 데이터
Vuex에 action을 발생시키기 위해 mapper 이용
화면 구성을 위해 template 정의
Two-Way Data Binding 기법으로 유저가 입력한 내용을 자동으로 데이터에 저장되게 처리
버튼 눌렀을 때 addItemAction 호출
Ch14에 ListComponent.vue 파일 생성
API를 import 받고 computed를 mapper 이용하여 정의
mapper를 이용하면, 함수 스타일로 등록하고 매개변수에 이용하고자 하는 state 값이 전달되어 편리성 있음
Ch14에 Vue 파일 Home.vue 제작
script 먼저 선언하여 두 개의 Component를 import받고, Vuex import 받음
데이터를 Vuex에 발생시키기 위한 action 함수를 mapper로 선언한 것
computed도 mapper 이용
component 등록
template으로 화면 구성
전체적으로 Home에 의해서 ListComponent와 AddComponent가 조합해서 나오는지 확인
이전에 생성한 ch13의 index.html, main.js, vue.config.js 파일 복사하여 이용
Home을 import 받고 store도 import시킴
두 개로 구분되는 것을 module로 등록 → 각각 구분되는 store라고 보면 됨
Vue cli 명령으로 실행
브라우저에 Component 출력

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

응용 실습 - 앱 구조 구축



한국기술교육대학교
온라인평생교육원

학습내용

- 실습 가이드
- 플러그인

학습목표

- **Ajax, Vuex, Vue-router**를 이용하여 애플리케이션의 구조를 구성할 수 있다.
- **플러그인**에 대해 설명할 수 있다.

실습 가이드

실습 가이드

1 응용 실습 Overview



응용 실습 Overview 특징

- 게시판 기능 구현
- Ajax를 이용한 서버 연동
- SPA 적용
- State Management 적용

실습 가이드

2 사용 기술

1

Vue.js

2

vue-router

3

Vuex

4

Axios

실습 가이드

실습 가이드

3 화면 구성

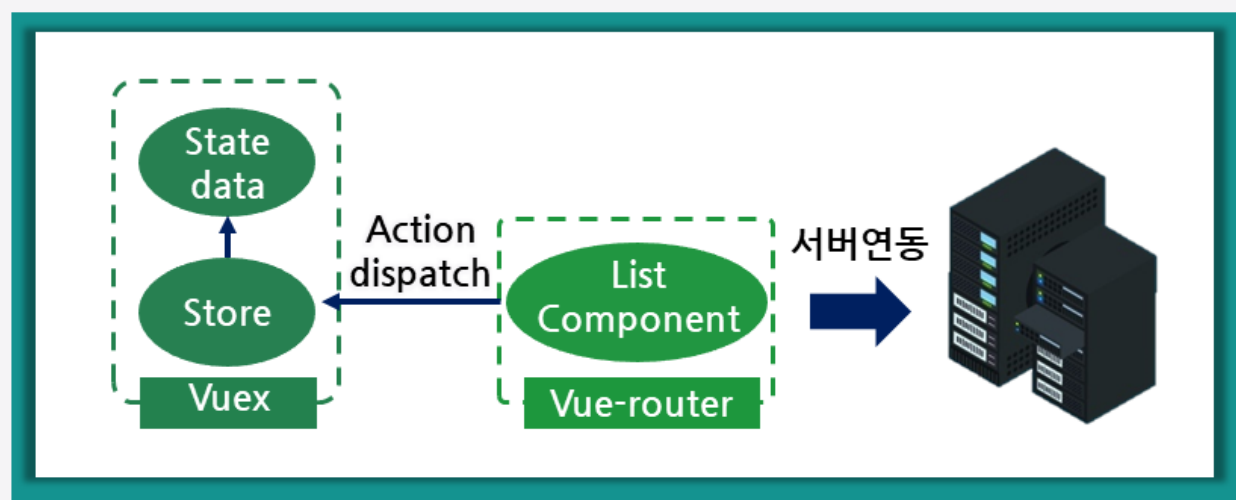
User List

Add User

Id	FirstName	LastName	Email	Action
1	gildong	hong	a@a.com	Delete Edit
2	hyunjin	ryu	b@b.com	Delete Edit

실습 가이드

4 앱 구조



플러그인

플러그인

1 플러그인

① 플러그인

... 플러그인은 Vue의 구성요소를 묶어 다른 곳에서 쉽게 재사용하기 위한 기법

Vue의 구성요소

- Component
- Directive
- Function
- Data ...

플러그인

1 플러그인

① 플러그인

... 여러 애플리케이션에서 공통으로 사용하기 위한 구성요소

... vue-router, Vuex 등은 플러그인으로 만들어짐

플러그인

플러그인

1 플러그인

② 플러그인 선언

- ... 플러그인은 `install` 함수를 가지는 Object Literal로 선언됨
- ... 플러그인이 이용되면 **`install` 함수가 자동 호출됨**
- ... `install` 함수의 첫 번째 매개변수로 플러그인을 이용하는 Vue의 application 객체가 전달됨

플러그인

1 플러그인

② 플러그인 선언

- ... 애플리케이션에서 Global하게 이용하기 위한 구성요소를 등록하여 사용함

```
export default {
  install: (app) => {
    app.component('my-plugin-component', MyHeader)
  }
}
```

플러그인

플러그인

1 플러그인

③ 플러그인 이용

→ 플러그인을 애플리케이션에서 사용하기 위해서는 application 객체의 use() 함수에 플러그인을 등록해야 함

```
var app = createApp(Home)
app.use(MyPlugin)
```

플러그인

1 플러그인

④ Component에서 플러그인 이용

```
<template>
  <div>
    <h3>Plugin Test1</h3>
    <my-plugin-component></my-plugin-component>
  </div>
</template>
```

플러그인

플러그인

1 플러그인

⑤ 플러그인에 Directive 등록

→ Global하게 이용하고자 하는 Directive를 플러그인에 등록

```
export default {
  install: (app) => {
    app.directive("my-font-size", (el) => {
      el.style.fontSize = 20 + "px";
    })
  }
}
```

플러그인

1 플러그인

⑤ 플러그인에 Directive 등록

→ Component에서 플러그인 이용

```
<template>
  <div>
    <p>Hello</p>
    <p v-my-font-size>Hello</p>
  </div>
</template>
```

플러그인

플러그인

1 플러그인

⑥ 플러그인 Option

- Option은 플러그인을 이용하는 곳에서 플러그인에 전달하는 데이터
- install 함수의 두 번째 매개변수로 전달됨

```
app.use(MyPlugin, {
  size: 30
})
```

플러그인

1 플러그인

⑥ 플러그인 Option

- 플러그인 내에서 Option 이용

```
export default {
  install: (app, options) => {
    app.directive("my-font-size", (el) => {
      el.style.fontSize = options.size + "px";
    })
  }
}
```

플러그인

플러그인

1 플러그인

⑦ Mixin

- 여러 Component에서 공통으로 적용되는 구성요소를 플러그인으로 제공
- 애플리케이션의 mixin에 등록해야 함

플러그인

1 플러그인

⑦ Mixin

- 특정 Component에서 사용 설정을 하지 않아도 모든 플러그인이 적용된 애플리케이션의 Component에 적용됨

```
export default {
  install: (app, options) => {
    app.mixin({
      data() {
        return {
          msg: 'Hello Plugin'
        }
      },
      created() {
        console.log("Plugin created....")
      }
    })
  }
}
```

플러그인

플러그인

1 플러그인

⑧ Provide

- 여러 Component에서 공통으로 적용되는 구성요소를 플러그인으로 제공
- application의 provide() 함수로 등록해야 함

플러그인

1 플러그인

⑧ Provide

- 필요한 Component에서 Inject 해서 사용하게 됨

```
export default {
  install: (app, options) => {
    const myClick = () => {
      console.log('myClick....')
    }
    app.provide('myClick', myClick)
  }
}
```

플러그인

플러그인

1 플러그인

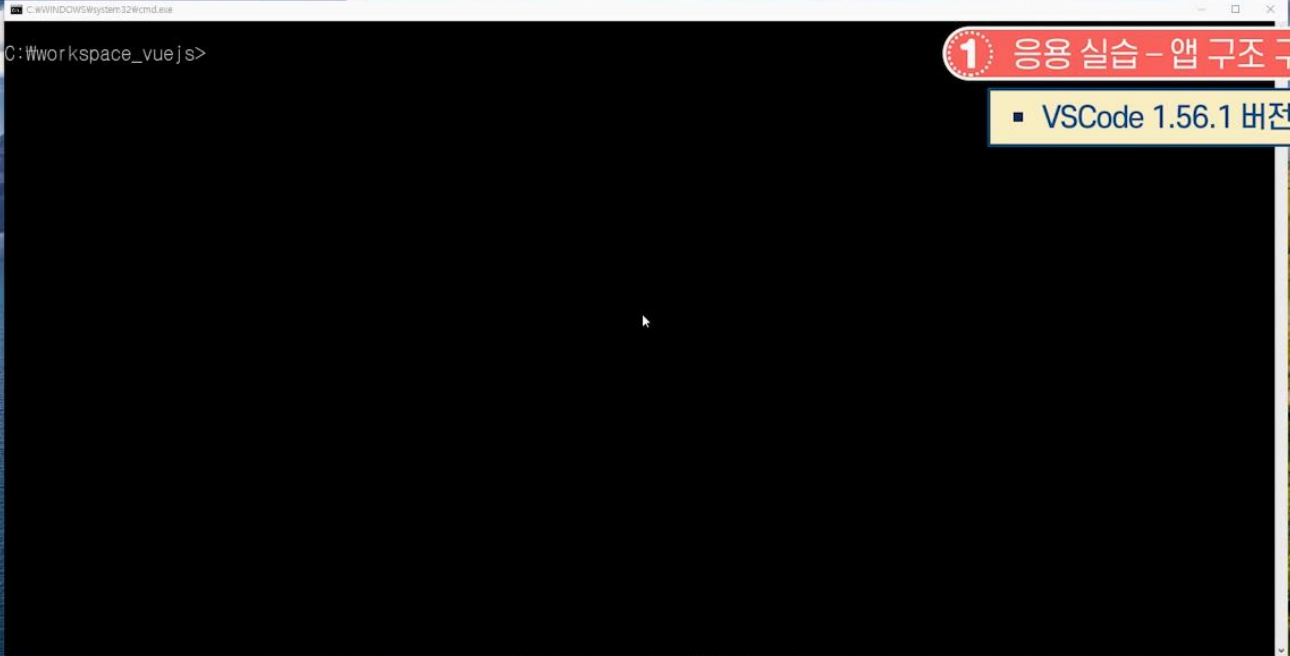
9 Provide 이용

```
import { inject } from 'vue'
myClick = inject('myClick')
```

실습 플러스

응용 실습 - 앱 구조 구축

실습 플러스



실습단계
vue create로 프로젝트 생성
방향키를 움직여서 vue3 선택
전체 프로젝트 Router로 화면 관리, 데이터는 Vuex 이용, 서버 연동 진행, 모듈 추가 설치
npm으로 vue-router 설치
npm으로 vuex 설치
HTTP를 통해서 서버 연동, 사용할 수 있는 모듈 설치
VS Code에서 지금 만들어진 프로젝트 열기
Vuex에서 데이터가 관리되기를 원함
src → store 폴더, listStore.js 파일 생성
axios import 받고 Object Literal 스타일로 store 선언
namespaced 선언, state 선언, 애플리케이션 전역 유지 데이터 users 데이터 유지
앱의 초기 데이터를 위한 서버 연동, 서버 데이터를 가지고 와서 애플리케이션 데이터로 구축할 때 사용하기 위한 getAllUsers 함수 등록
로컬에 있는 서버 이용

실습 플러스

실습단계
서버에 응답이 왔을 때 매개변수가 서버의 응답결과 값이 됨
action에서 commit 함수로 mutation에 있는 함수 명을 명시하여 호출되게 함, 두 번째 매개변수가 서버에서 받은 데이터
첫 번째 매개변수는 애플리케이션에서의 state 객체
Component에서 state 데이터를 쉽게 획득하기 위한 getter 선언
Component에서 id 값이 전달됨, 해당되는 유저를 전체 목록에서 찾아내서 return 시키기 위한 getter
Components 폴더 → ListComponent.vue 생성
Vuex를 이용하기 위한 import 먼저 선언하고 computed 함수 선언
listStore에서 namespace로 구분되는 store에 users 데이터 획득, template에 활용되면서 화면에 찍힘
Component의 LifeCycle 함수를 이용하여 액션 발생시킴
methods 옵션 추가, 비워둠
목록 화면을 구성하기 위한 테이블 구성
table 첫 줄에 thead Tag 추가
action, 비워둠
div Tag로 묶고 전체 화면이 Router에 의해 나올 수 있게 처리
script 부분 비움
style 제거함
main.js 수정
ListComponent를 등록
초기 값이 ListComponent가 나오도록 지정됨
Router 객체 생성
라우팅 정보가 들어가 있는 객체 선언
앱 생성
애플리케이션 객체에 router와 store 등록
중요! 제공된 실습 파일에서 다운받은 서버 이용
서버에 필요한 모듈 설치
Server.js 파일 : 서버 파일
node 명령으로 실행 → node server.js

실습 플러스

실습단계
vue-project 실행 → npm run serve 명령
브라우저에 결과 화면 출력

프론트엔드 개발자를 위한 Vue.js 시작하기

Vue.js </> 시작하기

응용 실습 - 앱 기능 구현



한국기술교육대학교
온라인평생교육원

학습내용

- 실습 가이드
- Vuetify

학습목표

- 응용 실습에 데이터 Add, Update, Delete 기능을 구현할 수 있다.
- Vuetify를 이용한 애플리케이션의 화면 구성 방법에 대해 설명할 수 있다.

실습 가이드

실습 가이드

1 Add 기능 구현

1 유저 입력 데이터 획득

2 서버 전송

User Add

Email address:

First Name:

Last Name:

실습 가이드

2 Update 기능 구현

1 유저 입력 데이터 획득

2 State에서 데이터 획득, 출력

3 Update 데이터 서버 전송

실습 가이드

실습 가이드

2 Update 기능 구현

User Update

Email address:

First Name:

Last Name:

실습 가이드

3 Delete 기능 구현

User List

Add User					
Id	FirstName	LastName	Email	Action	
1	gildong	hong	a@a.com	Delete	Edit
2	hyunjin	ryu	b@b.com	Delete	Edit

Vuetify

Vuetify

1 Vuetify 기본

1 Vuetify



Vuetify 특징

- Material Component를 제공하기 위한 UI Library
- 수려한 화면을 구성하기 위한 Component 및 Directive를 제공해 줌

Vuetify

1 Vuetify 기본

1 Vuetify



Vuetify 특징

- Vue.js 3 버전에서 사용하려면 Vuetify 3 버전을 이용해야 함



Vuetify

Vuetify

1 Vuetify 기본

2 Vuetify 설치

```
>vue add vuetify
```

Vuetify

1 Vuetify 기본

2 Vuetify 설치

- ... 3 버전의 Vue.js 프로젝트가 생성되어야 함
- ... Vue CLI 명령으로 프로젝트에 Vuetify를 설치함
- ... Vuetify 설치 시 선택사항을 제시하며 V3(Alpha)를 선택해야 함

```
? Choose a preset:
  Default (recommended)
  Preview (Vuetify 3 + Vite)
  Prototype (rapid development)
> V3 (alpha)
  Configure (advanced)
```


Vuetify

Vuetify

1 Vuetify 기본

③ Vuetify 객체 생성

- createVuetify() 함수로 Vuetify 객체를 생성함
- createVuetify() 함수에 Vuetify에서 제공하는 Component, Directive를 등록함

Vuetify

1 Vuetify 기본

③ Vuetify 객체 생성

```
import '@mdi/font/css/materialdesignicons.css'
import 'vuetify/lib/styles/main.sass'
import { createVuetify } from 'vuetify'
import * as components from 'vuetify/lib/components'
import * as directives from 'vuetify/lib/directives'

const vuetify = createVuetify({
  components,
  directives,
})
```

Vuetify

Vuetify

1 Vuetify 기본

4 Vuetify 객체 적용

- Vuetify는 Plugin으로 제공되는 라이브러리임
- Vue 애플리케이션에 use() 함수로 적용함

```
var app = createApp(Home)
app.use(vuetify)
app.mount('#app')
```

Vuetify

1 Vuetify 기본

5 Component에서 이용

- Vuetify에서 제공되는 Component는 v- 로 시작하는 태그명으로 이용됨

```
<template>
  <div>
    <h3>Vuetify Test</h3>
    <v-btn flat color="error"> Error </v-btn>
  </div>
</template>
```

ERROR

Vuetify

Vuetify

2 Vuetify Component

1 Avatar

- v-avatar Component 이용
- v-img Component로 지정한 이미지를 아바타 모양으로 출력

Vuetify

2 Vuetify Component

1 Avatar

- 사각형 이미지를 지정해도 기본으로 둥근 모양으로 출력함

```
<v-avatar size="large">
  <v-img src="https://cdn.vuetifyjs.com/images/lists/2.jpg"></v-img>
</v-avatar>
```



Vuetify

Vuetify

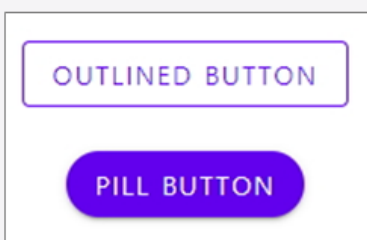
2 Vuetify Component

2 Button

→ v-button Component 이용

→ variant, color, rounded 속성을 이용해 다양한 버튼 이용

```
<v-btn variant="outlined" color="primary"> Outlined Button </v-btn>
<v-btn rounded="pill" color="primary"> Pill Button </v-btn>
```



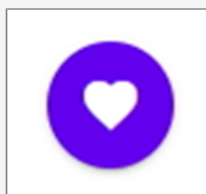
Vuetify

2 Vuetify Component

3 Icon Button

→ v-button Component에 icon 속성 이용

```
<v-btn icon="mdi-heart" color="primary" />
```



Vuetify

Vuetify

2 Vuetify Component

4 Rating

→ v-rating Component 이용

```
<v-rating
  v-model="rating"
  bg-color="orange-lighten-1"
  color="blue"
></v-rating>
```



Vuetify

2 Vuetify Component

5 List

→ v-list로 목록 출력

→ v-list 하위에 v-list-item으로 각 항목 출력

Vuetify

Vuetify

2 Vuetify Component

5 List

→ v-list-item 하위에 v-list-item-avatar, v-list-item-content, v-list-item-icon Component로 항목 구성

Vuetify

2 Vuetify Component

5 List

```
<v-list subheader>
  <v-subheader>Recent chat</v-subheader>
  <v-list-item v-for="chat in recent" :key="chat.title">
    <v-list-item-avatar>
      <v-img :alt="'${chat.title} avatar'" :src="chat.avatar"></v-img>
    </v-list-item-avatar>
    <v-list-item-content>
      <v-list-item-title v-text="chat.title"></v-list-item-title>
    </v-list-item-content>
    <v-list-item-icon>
      <v-icon :color="chat.active ? 'deep-purple accent-4' : 'grey'">
        mdi-message-outline
      </v-icon>
    </v-list-item-icon>
  </v-list-item>
</v-list>
```



Jason Oner



Mike Carlson



Cindy Baker



Ali Connors

실습 플러스

응용 실습 - 앱 기능 구현

실습 플러스

vue-project - Visual Studio Code

1 앱 기능 구현 실습

■ VSCode 1.56.1 버전 사용

실습단계
이전에 만들어 놓은 프로젝트 실습에 전체 목록과 관련된 것을 추가, 제거, 수정
Vuex에 store 부분 추가
store → listStore.js 선택
import 받은 이유는? route로 화면을 조정하기 위함
한 명의 유저를 등록하기 위한 addUserAction 함수 추가
입력되어 있는 데이터를 서버에 post 방식으로 전송하고 response의 status 값을 비교하여 이전 화면으로 화면 전환
update와 관련된 action 생성
update → put 방식 사용
URL 수정하고 뒤에 user의 아이디를 알려주어야 함
매개변수를 user가 아니라 id 값으로 받아서 서버에 전달
components 폴더에 EditComponent.vue 파일 생성
입력되어 있는 데이터를 저장하기 위한 변수 선언
EditComponent가 출력된 router의 패스 정보

실습 플러스

실습단계
함수 명 수정, updateUserAction이 실행되도록 준비
update 상황일 경우, 데이터가 출력될 수 있게 computed 정의
mounted 함수 이용, 파라미터 값에 id가 정의가 되어있으면 그 값을 먼저 얻어야 함
isAdd 값이 true인 경우 title 문자를 Add로 지정, false인 경우 Update로 지정
user 입력을 받기 위한 input 준비
두 번째 입력받는 값은 firstName으로 지정
마지막 입력받는 값은 lastName으로 지정
버튼이 클릭되면 addUser라는 함수 호출되게 지정
ListComponent.vue의 methods 부분에 함수 추가
목록 화면에서 화면 전환을 목적으로 호출, route로 준비
template에 add로 넘어갈 수 있는 button 추가
tr Tag에 비어있는 td 항목에 delete와 update button 제공
Component 하나가 더 추가됨 → main.js에 import 받음
path 조건이 add로 들어오면 EditComponents가 실행되게 함
routes 조건 등록에 따라 화면 조정
서버가 구동 되어있는 상태에서 Vue 프로젝트 실행
현재 에러로 인해 addUserAction 부분을 '_'로 처리
route 객체가 아니라 store 객체
methods에 deleteUser가 호출됐을 때 store action을 발생시키고자 하는 것이므로 \$ 추가
Vue 명령으로 실행
브라우저에 Component 출력